Daffodil International University
Department of Software Engineering

System Design Document

Unit 1: Identity & Access

TrustBazaar: A Verified Peer-to-Peer Auction and Marketplace Platform for Used Goods

S. M. Nihal Ahmed (232-35-002)
Afrim Hossen Khan (232-35-076)
Sabikun Nahar Sinthia (232-35-475)

Status	Draft for team review
Version	0.1, 14 October 2026
Implements	SRS v2.0 (`docs/srs.tex`)
Database schema	`docs/design/sql/01_identity_access.sql`, tested on PostgreSQL (46 checks, Appendix B)

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Planned order of units . . . 3
1.3 Scope of this unit . . . 3
1.4 Requirements covered . . . 3

2 Design Conventions for All Units **4**

3 Unit Overview **5**
3.1 Responsibilities . . . 5
3.2 Components . . . 5
3.3 Libraries . . . 6

4 Interfaces **6**
4.1 Request authentication and policies . . . 6
4.2 HTTP API . . . 7
4.3 Ports provided to other units . . . 9
4.4 Events published . . . 9
4.5 Ports required from other units . . . 10

5 Data Design **10**
5.1 Schemas . . . 10
5.2 Tables . . . 10
5.3 Why the important rules live in the database . . . 13
5.4 Redis keys . . . 13
5.5 Encryption and keys . . . 14

6 Behaviour **15**
6.1 Registration . . . 15
6.2 Sign-in, MFA and refresh . . . 15
6.3 Step-up authentication . . . 17
6.4 Password reset and change . . . 17
6.5 Contact changes and MFA . . . 17
6.6 KYC submission . . . 18
6.7 KYC review, revocation and blacklist . . . 18
6.8 Protected files . . . 19
6.9 Four-eyes approvals . . . 20
6.10 Suspension, closure and erasure . . . 20

6.11 Staff accounts 20

7 Security Design **21**

7.1 Tokens and cookies 21

7.2 Threats and how the design answers them 21

8 Parameters **22**

8.1 SRS parameters used 22

8.2 Design values introduced by this unit 22

9 Testing **23**

10 Notes for Integration **23**

A Traceability **24**

B Database Schema and Tests **25**

B.1 Test results 25

B.2 Schema 26

1 Introduction

1.1 Purpose

This document is the first unit of TrustBazaar's System Design Document (SDD). It specifies how the *Identity & Access* unit meets its requirements in the Software Requirements Specification, version 2.0 ("the SRS"). It covers the unit's components, interfaces, database schema, behaviour and security mechanisms. The SDD is being written one unit at a time; when all units are designed, they will be connected into the full system architecture.

This unit is designed first because every other unit depends on it. Every request must answer three questions before anything else happens: who is calling, is that person verified, and what are they allowed to do. This unit also establishes the conventions that all later units follow (Section 2), and owns two shared mechanisms: the audit log and four-eyes approvals.

1.2 Planned order of units

1. **Identity & Access** (this document).
2. Order, Payment & Escrow.
3. Auction Engine; Listings, Search & Fixed-Price Purchase (in parallel).
4. Delivery & Handover.
5. Game-Account Escrow.
6. Chat & Notifications.
7. Disputes, Trust & Safety, and Staff Tools.
8. Frontend: application structure, routing, API client and the screens of every unit, built on the APIs designed above.
9. Integration: full system architecture, combined data model and deployment view.

1.3 Scope of this unit

In scope: registration and phone/e-mail verification; sign-in, MFA, sessions and refresh; step-up authentication; password reset; contact changes; profile and consent records; account closure and erasure; KYC submission, review and revocation; the NID blacklist; staff accounts; four-eyes approvals; the audit log; and the request-authentication and authorisation guards used by every endpoint in the system.

Out of scope, owned by later units: delivering SMS and e-mail (Notifications unit; this unit calls a port, Section 4.5); deciding sanctions (Trust & Safety unit, which calls this unit's commands to suspend an account or blacklist an identity); parameter versioning and the administrator's parameter screens (Staff Tools unit); and Pro Seller status (Monetization, in the Order unit).

1.4 Requirements covered

The unit implements the requirements below; Appendix A maps each one to the design element that satisfies it.

- **Functional:** FR-ACC-01 to FR-ACC-11; FR-KYC-01 to FR-KYC-12; FR-TNS-06; FR-OPS-03 to FR-OPS-07 (staff accounts, four-eyes approval, conflicts, staff action logging); FR-PRV-01, FR-PRV-02, FR-PRV-04, FR-PRV-05; FR-NTF-06 (as event source).
- **Security:** SEC-AUTH-01 to SEC-AUTH-12; SEC-SES-01 to SEC-SES-07; SEC-ACC-01 to SEC-ACC-06; SEC-DAT-01 to SEC-DAT-09 and SEC-DAT-11; SEC-INP-04 (KYC uploads); SEC-ABU-01, SEC-ABU-03 (authentication limits); SEC-LOG-01 to SEC-LOG-05.

- **Other:** IF-UI-01, IF-UI-02, IF-UI-09 (staff views); IF-SW-05, IF-SW-06; NFR-USE-02; DR-01, DR-02, DR-09, DR-10.

2 Design Conventions for All Units

These conventions are set here because this is the first unit; every later unit follows them unless its own design document records a justified exception.

ID	Convention	Rule
DC-1	Modular monolith	One NestJS backend (constraint C-1). Each unit is a group of NestJS modules. Units interact only through exported service interfaces (*ports*) and domain events, never by reading another unit's tables or importing its repositories. This keeps the four-container deployment (NFR-MNT-01) while allowing each unit to be designed, tested and later extracted on its own.
DC-2	Layers	Controller (HTTP or WebSocket, request validation) → application service (one method per use case, owns the database transaction) → repository (TypeORM). Business rules live in services and entities, not in controllers.
DC-3	HTTP API	JSON under `/api/v1`. Errors use RFC 9457 `application/problem+json` with a stable `type`, a user-safe `title` and the request's `correlationId` (SEC-INP-06). Lists use cursor pagination. Request bodies are validated by `class-validator` with `whitelist` and `forbidNonWhitelisted` on, so unknown fields are rejected (SEC-INP-01, SEC-ACC-04).
DC-4	Database	One PostgreSQL database, one schema per unit (this unit: `identity` and `audit`). Primary keys are random UUIDs (SEC-DAT-11). All times are `timestamptz`. Money is `bigint` poisha. Invariants that must never break are enforced by database constraints or triggers as well as in code. Schema changes are versioned migrations; TypeORM `synchronize` is used only on a developer's machine (NFR-MNT-04).
DC-5	Cross-unit references	A unit may hold a foreign key only to a unit designed *earlier* (in practice, to `identity.users` and `identity.staff_accounts`). References in the other direction are stored as plain UUIDs without a foreign key (for example `nid_blacklist.related_order_id`). This prevents cycles and keeps each schema loadable on its own.
DC-6	Events	Domain events are named `<unit>.<entity>.<past_tense>`. Events whose loss would break a requirement (security notices, KYC decisions, money movements) are written to the unit's *outbox* table in the same transaction as the change, and a background worker publishes them at least once. Consumers must therefore be idempotent. Other events may be emitted in-process after commit.
DC-7	Authorisation	A global guard denies every request by default. Each controller method declares its policy with a decorator (Section 4.1). A method with no policy cannot be reached.
DC-8	Audit	Every consequential action calls `AuditService.record()` inside the same database transaction, so the action and its audit entry commit or fail together (SEC-LOG-01).
DC-9	Configuration	Parameters (CP-nn) are read through a `ConfigPort`, never hard-coded (NFR-MNT-06). Secrets and keys come from environment variables and carry a version number (SEC-DAT-05).
DC-10	Time	Only the server clock is trusted (AS-2). Services receive an injected `Clock`, so deadlines can be tested without waiting.
DC-11	Shared security middleware	Security headers (`helmet`: HSTS, CSP with nonces, `nosniff`, `Referrer-Policy`, `Permissions-Policy`), CORS allow-list and cookie parsing are configured once in `main.ts` as part of this unit (SEC-WEB-01 to SEC-WEB-03).

ID	Convention	Rule
DC-12	Testing	Each unit has unit tests for its services, integration tests against real PostgreSQL and Redis (the CI service containers), negative authorisation tests for each endpoint (SEC-SDL-09) and, where the database enforces a rule, an SQL test script (Appendix B).

3 Unit Overview

3.1 Responsibilities

1. Create accounts only for people who have proven control of a Bangladeshi mobile number, and verify their e-mail.
2. Authenticate users and staff, issue and revoke sessions, and authenticate every HTTP and WebSocket request.
3. Verify identity through KYC, and guarantee that one NID belongs to at most one account.
4. Decide, for every request in the system, whether the caller may perform it (roles, KYC status, account status, step-up).
5. Keep the tamper-evident audit log and run four-eyes approvals for every unit.
6. Protect identity data: encrypt it, limit and log access to it, and erase it on schedule.

3.2 Components

Figure 1 shows the unit's components. The backend modules map to folders in `backend/src`: the existing `auth`, `users` and `kyc` modules, plus three new ones, `staff`, `audit` and `security`. The `security` module is shared infrastructure (guards, cryptography, rate limiting) that every other unit imports.

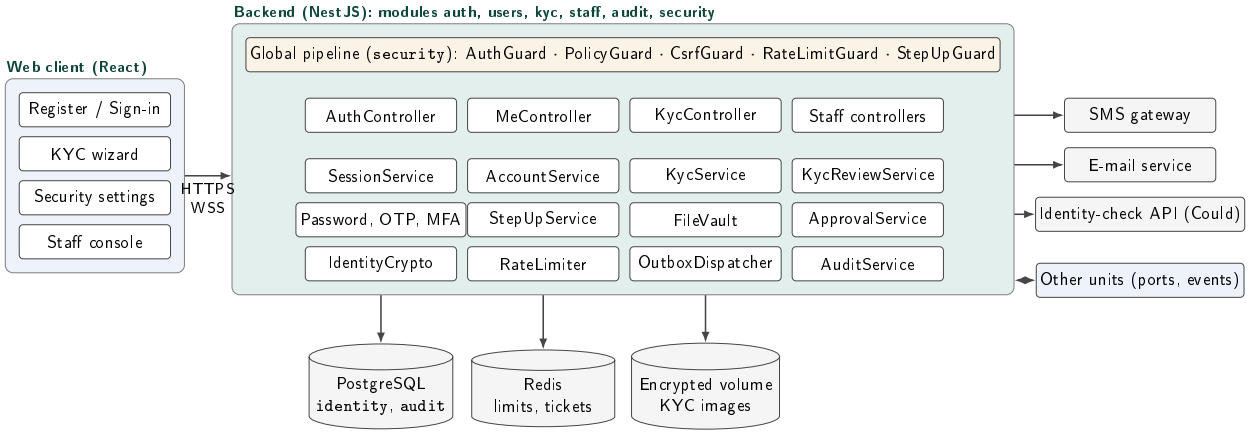


Figure 1: Components of the Identity & Access unit. SMS and e-mail are reached through the Notification port; in the academic deliverable they are logged test adapters (AS-4).

Component	Responsibility
AuthController	Registration, sign-in, MFA challenge, refresh, sign-out, sessions, step-up, password reset (Section 4.2).
MeController	Profile, contact changes, MFA enrolment, consents, account closure.
KycController	Liveness challenge, KYC submission, status, viewing one's own images.
Staff controllers	Staff sign-in, KYC review queue and decisions, approvals, blacklist proposals, conflict declarations; administrator staff management and audit-log view.

Component	Responsibility
SessionService	Creates sessions, rotates refresh tokens, detects reuse, ends sessions, records devices.
AccountService	Pending registrations, account creation, contact changes, closure and erasure.
Password / OTP / MFA	Argon2id hashing and policy checks; OTP generation, hashing and verification; TOTP enrolment, verification and recovery codes.
StepUpService	Issues and consumes single-use step-up grants bound to a session and an action.
KycService	Validates and stores submissions; computes blind indexes; flags duplicates, blacklist matches and age.
KycReviewService	Review queue, claiming, decisions, revocation proposals and the identity-claim guarantee.
FileVault	Re-encodes, encrypts, stores, serves and erases KYC images; issues signed view links.
ApprovalService	Four-eyes proposals and decisions for all units, with a handler registry.
IdentityCrypto	Field encryption, blind indexes and key versions.
RateLimiter	Shared Redis counters for every limited endpoint in the system.
AuditService	Writes and verifies the hash-chained audit log for all units.
OutboxDispatcher	Publishes outbox events and retries failures.

3.3 Libraries

The stack is fixed by constraint C-1. In addition, this unit uses the following established libraries, chosen so that no cryptography or parsing is written by hand (C-4).

Library	Use
`argon2`	Argon2id password hashing (SEC-AUTH-03). Replaces the `bcrypt` dependency currently in `backend/package.json`.
`otplib`	TOTP generation and verification (RFC 6238).
`libphonenumber-js`	Parsing and validating Bangladeshi mobile numbers into E.164 form.
`sharp`	Decoding, metadata stripping and re-encoding of uploaded images (SEC-DAT-08, SEC-INP-04).
`file-type`	Identifying uploads by their content, not their name (SEC-INP-04).
`helmet`, `cookie-parser`	Security headers and cookie handling (DC-11).
`Node.js crypto`	AES-256-GCM, HMAC-SHA-256, SHA-256 and secure random bytes.

4 Interfaces

4.1 Request authentication and policies

Every HTTP request and WebSocket connection passes through the same pipeline before it reaches a controller:

1. **RateLimitGuard** applies the endpoint's limit (Section 5.4); exceeding it returns `429` with `Retry-After`.
2. **AuthGuard** reads the access-token cookie and verifies the JWT, accepting only HS256 and a known key id (SEC-SES-01). It then loads the session row by its id (the `sid` claim) together with the principal's status, in one indexed query. The request is rejected unless: the session has not ended; it is within its idle and absolute limits; and the user or staff account is active (or suspended, with the restrictions below). A request is never authenticated by the token alone, so ending a session takes effect on the next request (SEC-SES-05). `last_seen_at` is updated at most once a minute.

3. **CsrfGuard**, for every state-changing method, checks the `Origin` header against the allow-list. It also checks that the `X-CSRF-Token` header equals the CSRF cookie and that both are an HMAC of the session id (a signed double-submit token, SEC-SES-04).
4. **PolicyGuard** enforces the method's declared policy (below). A method without a policy is unreachable (DC-7).
5. **StepUpGuard**, for methods marked with `@StepUp`, consumes a step-up grant (Section 6.3).

Policy decorator	Allows
`@Public()`	Anyone, including guests (browsing, registration, sign-in).
`@SignedIn()`	Any user with an active or suspended account.
`@Verified(action)`	A user who is `verified`, has a verified e-mail, an active (not suspended) account and no restriction for `action`, where `action` is one of `list`, `bid`, `buy` or `payout`. This is the server-side gate of FR-KYC-07, used by all transacting units through `IdentityQuery.assertCanTransact`.
`@Staff(role)`	An active staff account of the given role whose session satisfied MFA.
`@StepUp(action)`	In addition to the above, a fresh step-up grant for `action`.

A suspended user (FR-TNS-05) can still sign in, see their orders and complete paid ones. `@Verified` refuses every new listing, bid and purchase.

4.2 HTTP API

All paths are under `/api/v1`. "RL" names the rate-limit rule of Section 5.4. Request and response schemas will be generated as OpenAPI from the DTOs (NFR-MNT-05).

Method	Path	Policy	RL	Purpose (requirement)
Registration and sign-in				
`POST`	`/auth/register`	Public	register	Start registration; send OTP (FR-ACC-01, 03, 04)
`POST`	`/auth/register/verify`	Public	otp-verify	Verify OTP; create account (FR-ACC-02)
`POST`	`/auth/register/resend`	Public	otp-send	Resend OTP (SEC-AUTH-09)
`POST`	`/auth/email/verify`	Public	token	Verify e-mail link (FR-ACC-02)
`POST`	`/auth/signin`	Public	signin	Password step (FR-ACC-05)
`POST`	`/auth/signin/mfa`	MFA ticket	otp-verify	Second factor (FR-ACC-06)
`POST`	`/auth/refresh`	Refresh cookie	refresh	Rotate tokens (SEC-SES-02)
`POST`	`/auth/signout`	SignedIn	—	End this session (SEC-SES-05)
Sessions, step-up and passwords				
`GET`	`/auth/sessions`	SignedIn	—	List sessions (FR-ACC-08)
`DELETE`	`/auth/sessions/{id}`	SignedIn	—	End one session
`POST`	`/auth/sessions/end-others`	SignedIn	—	End all other sessions
`POST`	`/auth/step-up/start`	SignedIn	otp-send	Begin step-up for an action
`POST`	`/auth/step-up/verify`	SignedIn	otp-verify	Obtain a step-up grant (SEC-AUTH-08)
`POST`	`/auth/password/forgot`	Public	reset	Send reset link or OTP (FR-ACC-07)
`POST`	`/auth/password/reset`	Public	token	Set new password (SEC-AUTH-10)
`POST`	`/auth/password/change`	StepUp	—	Change password (SEC-AUTH-11)

Method	Path	Policy	RL	Purpose (requirement)
Profile, contacts, MFA, consent, closure				
GET	/me	SignedIn	—	Own account
PATCH	/me	SignedIn	—	Display name, area (FR-ACC-10)
GET	/users/{id}/profile	Public	search	Public profile (FR-ACC-10)
POST	/me/phone/change	StepUp	otp-send	OTP to new number (FR-ACC-09)
POST	/me/phone/change/verify	SignedIn	otp-verify	Confirm new number
POST	/me/email/change	StepUp	token	Link to new e-mail (FR-ACC-09)
POST	/me/mfa/totp	SignedIn	—	Start TOTP enrolment (FR-ACC-06)
POST	/me/mfa/totp/confirm	SignedIn	otp-verify	Confirm; issue recovery codes
DELETE	/me/mfa/totp	StepUp	—	Turn MFA off
POST	/me/mfa/recovery-codes	StepUp	—	Regenerate recovery codes
GET, POST	/me/consents	SignedIn	—	Optional consents (FR-PRV-02)
POST	/me/close	StepUp	—	Close account (FR-ACC-11)
KYC (user)				
GET	/kyc/status	SignedIn	—	Status and reason (IF-UI-02)
GET	/kyc/liveness-challenge	SignedIn	kyc	Random action (FR-KYC-02)
POST	/kyc/submissions	SignedIn	kyc	Submit documents (FR-KYC-01)
POST	/kyc/files/{kind}/view	SignedIn	—	Signed link to own image
GET	/files/v/{token}	Signed link	files	Stream a protected image (SEC-DAT-04)
Staff: operators				
POST	/staff/auth/signin, /staff/auth/mfa	Public	signin	Staff sign-in (SEC-AUTH-07)
POST	/staff/auth/mfa/enrol	Pending staff	—	First MFA enrolment
GET	/staff/kyc/queue	Staff(operator)	—	Oldest first, with flags (FR-OPS-01)
POST	/staff/kyc/submissions/{id}/claim	Staff(operator)	—	Take for review (SEC-ACC-06)
GET	/staff/kyc/submissions/{id}	Staff(operator)	—	Details (claimant only)
POST	/staff/kyc/submissions/{id}/files/{kind}/view	Staff(operator)	files	Signed link, reason required
POST	/staff/kyc/submissions/{id}/decision	Staff(operator)	—	Approve or reject (FR-KYC-06)
POST	/staff/users/{id}/kyc/revoke	Staff(operator)	—	Propose revocation (FR-KYC-10)
POST	/staff/blacklist	Staff(operator)	—	Propose blacklisting (FR-TNS-06)
POST	/staff/blacklist/{id}/remove	Staff(operator)	—	Propose removal
GET	/staff/approvals	Staff(any)	—	Open proposals for my role
POST	/staff/approvals/{id}/decision	Staff(required role)	—	Approve or reject (FR-OPS-05)
POST	/staff/conflicts	Staff(any)	—	Declare a conflict (FR-OPS-06)
Staff: administrators				
GET, POST	/admin/staff	Staff(admin)	—	List, create staff (FR-OPS-04)
POST	/admin/staff/{id}/disable	Staff(admin)	—	Disable a staff account

Method	Path	Policy	RL	Purpose (requirement)
POST	/admin/staff/{id}/mfa-reset	Staff(admin)	—	Propose MFA reset (four-eyes)
GET	/admin/audit, /admin/audit/verify	Staff(admin)	—	Read and verify the log (SEC-LOG-04)

The first administrator is created by a command run inside the backend container (`npm run staff:bootstrap`). The command refuses to run if any administrator already exists, and the database permits only one account without a creator (FR-OPS-04).

4.3 Ports provided to other units

Other units use this unit only through the following interfaces, exported from the `auth`, `kyc`, `staff` and `audit` modules.

```
interface IdentityQuery {
  getPrincipal(userId: string): Promise<{ id: string; displayName: string;
      kycStatus: KycStatus; accountStatus: AccountStatus; emailVerified: boolean }>;
  assertCanTransact(userId: string, action: 'list' | 'bid' | 'buy' | 'payout'): Promise<void>; // throws
  getPublicProfile(userId: string): Promise<PublicProfile>;          // FR-ACC-10 fields only
  getVerifiedPhone(userId: string): Promise<string>;                 // payout wallet match, FR-PAY-17
  getKycLegalName(userId: string, purpose: 'bank_payout_check'):     // decrypts; audited
      Promise<string>;
  hasConflict(staffId: string, userId: string): Promise<boolean>;    // FR-OPS-06
}
interface IdentityCommands {               // called by Trust & Safety; each call is audited
  suspendAccount(userId: string, reason: string, byStaffId: string): Promise<void>;  // ends sessions
  reinstateAccount(userId: string, approvalId: string): Promise<void>;
  endAllSessions(userId: string, reason: SessionEndReason): Promise<void>;
  proposeBlacklist(userId: string, source: BlacklistSource, relatedOrderId: string | null,
      reason: string, byStaffId: string): Promise<Approval>;
}
interface ApprovalService {                // four-eyes for every unit (FR-OPS-05)
  propose(p: { action: string; targetType: string; targetId: string; payload: object;
      reason: string; requiredRole: StaffRole; byStaffId: string }): Promise<Approval>;
  decide(id: string, byStaffId: string, decision: 'approved' | 'rejected', note?: string): Promise<void>;
  registerHandler(action: string, h: (a: Approval, tx: EntityManager) => Promise<void>): void;
}
interface AuditService {
  record(e: { actorType: ActorType; actorId?: string; action: string; objectType?: string;
      objectId?: string; before?: object; after?: object; reason?: string; approvalId?: string },
      tx: EntityManager): Promise<void>;     // same transaction as the action (DC-8)
  verifyChain(): Promise<{ intact: boolean; firstBadId?: number }>;
}
interface RateLimiter { consume(rule: string, subject: string): Promise<void> }  // throws 429
```

4.4 Events published

All events below go through the outbox (DC-6) and carry the user or staff id, the time and a correlation id. The Notifications unit turns the security events into the notices required by FR-NTF-06.

Event	Consumers and purpose
`identity.user.registered`	Notifications (welcome, e-mail verification link).
`identity.email.verified`	None required; available for analytics.
`identity.signin.new_device`	Notifications: SMS and e-mail alert (FR-NTF-06).
`identity.password.changed`	Notifications: alert (FR-NTF-06).
`identity.contact.changed`	Notifications: alert to the *old* number or e-mail (FR-ACC-09).
`identity.mfa.enabled`, `identity.mfa.disabled`	Notifications: alert (FR-NTF-06).

Event	Consumers and purpose
`identity.session.reuse_detected`	Notifications: alert; Trust & Safety: signal.
`identity.kyc.submitted`	Notifications: confirmation with expected review time (FR-KYC-12).
`identity.kyc.verified`, `identity.kyc.rejected`	Notifications (FR-NTF-03); Listings may publish drafts.
`identity.kyc.revoked`	Listings: hide active listings; Auctions and Orders: refuse new bids and purchases (FR-KYC-10).
`identity.account.suspended`, `identity.account.reinstated`	Listings: hide or restore; Notifications (FR-TNS-03, FR-TNS-05).
`identity.account.closed`	All units: stop processing for the user.

4.5 Ports required from other units

Port	Provided by, and use
`NotificationPort`	Notifications unit. `sendSms(phone, template, vars)` for OTPs and security alerts; `sendEmail(address, template, vars)` for links. Until that unit exists, a test adapter writes messages to the log and a development mailbox, labelled as simulated (AS-4). OTPs are sent synchronously through this port rather than through the outbox, because the user is waiting for them.
`AccountClosureCheck[]`	Orders, Auctions, Payments and Disputes units each register a check. A check returns the reasons, if any, why the user cannot close the account yet (open order, unpaid win, open dispute, balance owed), as required by FR-ACC-11.
`ConfigPort`	Staff Tools unit (parameter store); initially environment configuration. Returns CP-nn values.
`IdentityCheckPort`	Optional pre-check provider (FR-KYC-11, Could). Returns advisory scores only.

5 Data Design

The complete schema is in `docs/design/sql/01_identity_access.sql` and is reproduced in Appendix B. It has been loaded into PostgreSQL and checked by a test script of 46 cases (Appendix B.1). This section explains the structure and the rules the database itself enforces.

5.1 Schemas

The unit owns two schemas: `identity` (20 tables) for accounts, credentials, sessions, KYC, staff and approvals, and `audit` (one table) for the system-wide audit log. Other units will reference `identity.users(id)` and `identity.staff_accounts(id)` by foreign key; this unit references no other unit (DC-5).

5.2 Tables

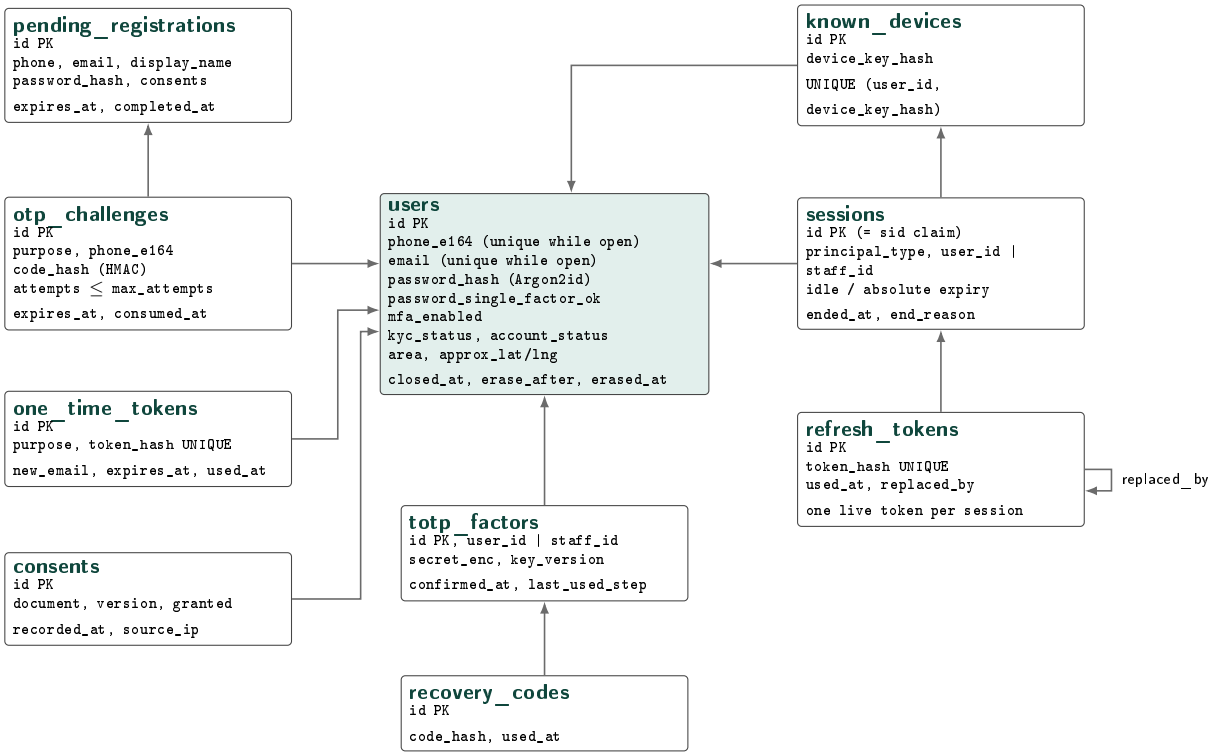


Figure 2: Accounts and authentication tables (schema `identity`). Arrows point from the referencing table to the referenced one. `sessions` and `totp_factors` also reference `staff_accounts` (Figure 3).

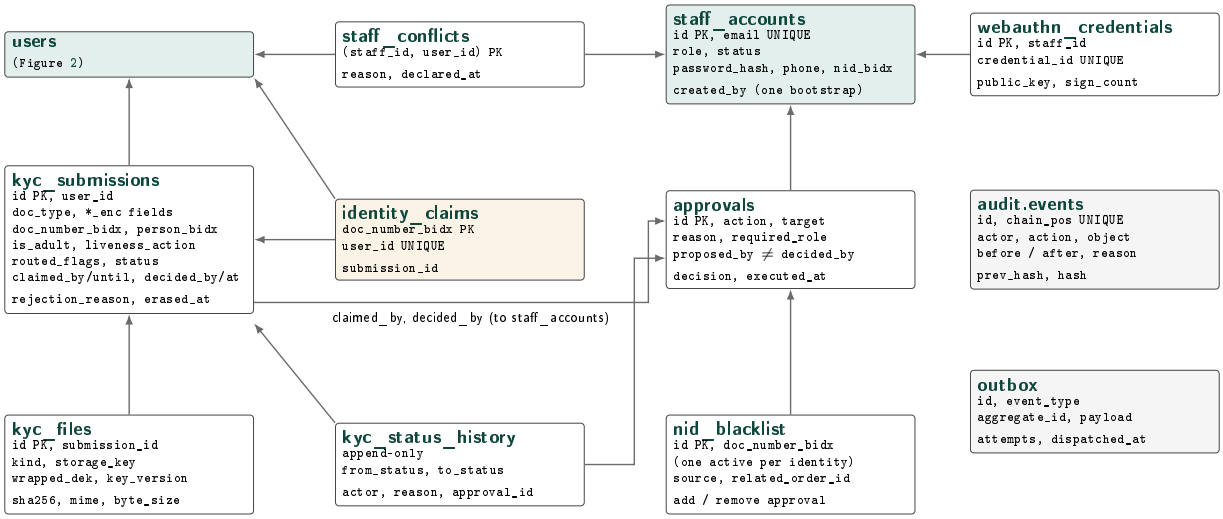


Figure 3: KYC, staff, approvals and audit tables. `identity_claims` (shaded) is what makes "one identity, one account" a database guarantee. `audit.events` and `outbox` have no foreign keys, so any unit can write to the audit log.

Table	Holds	Rules enforced by the database
`users`	One row per account; created only after phone OTP.	Mobile number and e-mail unique among accounts that are not closed (case-insensitive e-mail); Bangladeshi mobile format; Argon2id hashes only; a password under 15 characters only while MFA is on; KYC status changes only along permitted edges (trigger); personal fields may be emptied only after erasure of a closed account.
`pending_registrations`	Registration details until the OTP is verified.	Password must meet the 15-character rule (no MFA yet); age and terms confirmed.
`otp_challenges`	Hashed OTPs for registration, step-up, phone change and reset.	Attempts never exceed the maximum; registration OTPs belong to a pending registration; other OTPs belong to a user.
`one_time_tokens`	Hashed e-mail verification, e-mail change and reset tokens.	Token hash unique; e-mail-change tokens carry the new address.
`consents`	Terms, privacy and optional consents, with version and time.	— (append in practice; history kept).
`known_devices`	Devices previously used by a user (hash of a device cookie).	Unique per user and device.
`sessions`	One sign-in on one device, for a user or a staff member.	Exactly one principal; idle expiry $\leq$ absolute expiry; an end reason whenever ended.
`refresh_tokens`	Rotating refresh tokens of a session.	At most one unused token per session; used tokens record their successor.
`totp_factors`, `recovery_codes`	Encrypted TOTP secrets; hashed recovery codes.	One active factor per principal.
`webauthn_credentials`	Optional staff passkeys.	Credential id unique.
`kyc_submissions`	One row per submission, encrypted document details, review state.	One open submission per user; a rejection needs a reason code; an underage submission cannot be approved; decision fields set together; encrypted fields removable only by erasure.
`kyc_files`	Encrypted images of a submission.	JPEG, PNG or WebP only; at most 10 MB; one file per kind; an erased file has no key.
`identity_claims`	The identity that each verified account holds.	Primary key on the blind index and unique user: one NID, one account (FR-KYC-04).
`kyc_status_history`	Every KYC status change.	Append-only (trigger and grants).
`staff_accounts`, `staff_conflicts`	Operators and administrators; declared conflicts.	Argon2id only; only one account may lack a creator (the bootstrap administrator).
`approvals`	Four-eyes proposals for all units.	Proposer $\neq$ decider; one open proposal per action and target; execution only after approval.
`nid_blacklist`	Banned identities (blind index).	One active entry per identity; addition and removal each reference an approval.
`outbox`	Events awaiting publication.	Event names belong to this unit.
`audit.events`	System-wide audit log.	Append-only (trigger and grants); each row hashed with its predecessor; chain order assigned under a lock.

5.3 Why the important rules live in the database

Four guarantees are enforced by the database as well as by the service code, because a bug or a race in application code must not be able to break them:

1. **One identity, one account.** Approving a KYC submission inserts its blind index into `identity_claims` in the same transaction. Two operators approving two accounts with the same NID at the same moment cannot both succeed; the second transaction fails on the primary key and is rolled back.
2. **Linear refresh rotation.** The partial unique index allows only one unused refresh token per session, so two concurrent refreshes cannot both mint a valid successor. The successor link is checked at commit (a deferrable foreign key), so the old token can be marked used before its successor is inserted.
3. **Four-eyes.** The check `decided_by <> proposed_by` makes self-approval impossible even through a faulty endpoint. The checks use `IS NOT DISTINCT FROM` because a plain comparison with a NULL decision evaluates to NULL, which PostgreSQL treats as passing. The test script caught exactly this mistake in the first draft.
4. **Tamper-evident audit.** A trigger computes each entry's hash from the previous entry under an advisory lock. Ids are allocated before the lock is taken, so the chain is ordered by a separate `chain_pos` assigned inside it. Verification therefore stays correct under concurrent writers, tested with 8 parallel connections. `audit.verify_chain()` returns the first entry whose hash does not match.

5.4 Redis keys

Redis holds only short-lived and reconstructible data (NFR-REL-01). Losing it resets counters and expires pending tickets: users repeat a step, but no account, session or decision is lost. OTP attempt limits remain enforced by `otp_challenges.attempts` in PostgreSQL even if Redis counters are lost.

Key	Value and use	Lifetime
`rl:{rule}:{subject}`	Fixed-window counter, updated atomically by a Lua script (SEC-ABU-01, SEC-ABU-03).	Window of the rule.
`signin:fail:{account}`	Failed sign-in count and delay-until time (SEC-AUTH-04).	CP-04 window.
`otp:send:{phone}:h, :d`	OTP sends per number per hour and per day (CP-05).	1 hour, 1 day.
`mfa:ticket:{hash}`	Principal and device waiting for the second factor.	DV-1 (5 min).
`stepup:{sid}:{action}`	Hash of a single-use step-up grant, consumed with `GETDEL`.	DV-2 (5 min).
`liveness:{challengeId}`	User and the random action for the selfie.	DV-3 (10 min).

Rule	Applies to	Limit
`signin`	sign-in (user and staff)	CP-04: 5 failures per account per 15 min, then progressive delay; 50 per source address per 15 min.
`register`	registration	5 per source address per hour.
`otp-send`	any OTP send	CP-05: 3 per number per hour, 10 per day; 10 per source address per hour.
`otp-verify`	any OTP or TOTP entry	CP-05: 5 attempts per challenge; 20 per account per hour.

Rule	Applies to	Limit
`reset`	forgotten password	3 per account per hour; 10 per source address per hour.
`token`	link-token use	20 per source address per hour.
`refresh`	token refresh	60 per session per hour.
`kyc`	KYC challenge and submission	10 per user per day (resubmission limit CP-34 enforced separately).
`files`	signed image links	120 per staff member per hour (alert at 60, SEC-LOG-05).
`search`	public profile reads	CP-35: 60 per minute per source address.

5.5 Encryption and keys

Fields. Document number, full name, date of birth and TOTP secrets are encrypted with AES-256-GCM. Each value is stored as one byte of key version, a 12-byte random IV, the ciphertext and the 16-byte tag. The *associated data* is the table, column and row id, so a ciphertext copied into another row or column fails to decrypt.

Blind indexes. `doc_number_bidx` is HMAC-SHA-256 under a separate key, over the document type and the number reduced to digits. `person_bidx` is the same HMAC over the normalised full name (lower case, single spaces, Latin script as printed) and the date of birth. It is a second, *non-blocking* signal (`possible_duplicate`), because one person may hold NID numbers in different formats, which the number index alone would treat as different people.

Images. Each KYC image gets a fresh 256-bit data key and is encrypted with AES-256-GCM (associated data: the file id). The data key is wrapped with the key-encryption key, also AES-256-GCM. Files are written with random names and mode `0600` to a Docker named volume, `kyc_vault`, mounted only into the backend container. This keeps within the four-container limit (NFR-MNT-01). Erasure deletes the file and sets `wrapped_dek` to NULL (crypto-shredding). A copy of the file in a volume backup can then no longer be decrypted once the database backups that still contain the wrapped key have also expired.

Secret	Purpose	Rotation
`FIELD_KEY_V{n}`	Field encryption (KYC details, TOTP secrets).	New version for new writes; background re-encryption; old version kept until no row uses it.
`BIDX_KEY`	Blind indexes (documents, persons, staff NIDs).	Rotation requires recomputing every index from decrypted values in one maintenance job; rotate only on compromise.
`KYC_KEK_V{n}`	Wraps per-file data keys.	Re-wrap data keys only; files are not re-encrypted.
`JWT_KEY_V{n}`	HS256 access tokens (`kid` header).	New key signs immediately; previous key accepted for CP-01.
`CSRF_KEY`, `LINK_KEY`	HMAC for CSRF tokens and signed file links.	Any time; outstanding tokens expire within minutes.
`OTP_PEPPER`	HMAC of OTPs, so that 6-digit codes cannot be brute-forced from a database copy.	On compromise; pending OTPs become invalid.

All secrets are read from environment variables at start-up, are never logged, and the backend refuses to start if any is missing or shorter than 32 bytes (SEC-DAT-05).

6 Behaviour

Each flow below is one application-service method running in one database transaction unless stated otherwise. Every flow writes its audit entry inside that transaction (DC-8). "Outbox" means an event row is written in the same transaction (DC-6).

6.1 Registration

1. `POST /auth/register` with mobile number, e-mail, display name, password, the accepted terms and privacy versions, and the age confirmation. The mobile number is parsed to E.164 and must be a Bangladeshi mobile number. The password must have at least 15 characters (no MFA exists yet) and must not be in the blocklist of common, breached and context-specific passwords (SEC-AUTH-01, SEC-AUTH-02). The blocklist is a file bundled with the backend, so no password or hash prefix is sent to any outside service.
2. The password is hashed with Argon2id and a `pending_registrations` row is created.
   - If the number and e-mail are both new, an OTP challenge is created and the code is sent by SMS.
   - If the number already belongs to an open account, no code is sent. Instead, that account's owner receives an SMS: "someone tried to register with your number".
   - Either way, the response is identical: `202` with a registration id and a resend time (SEC-AUTH-05, SEC-AUTH-06).
3. `POST /auth/register/verify` with the code. The code is compared in constant time with its HMAC; a failure increments `attempts`, and the database caps attempts at the maximum. On success, one transaction:
   - creates the `users` row and the `consents` rows;
   - marks the challenge consumed and the registration completed;
   - creates an e-mail verification token;
   - writes `identity.user.registered` to the outbox;
   - audits `account.registered`.

   The user is signed in (Section 6.2).
4. If the e-mail turns out to belong to another open account, the unique index rejects the insert. The response is then a generic "could not create an account with these details; if you already have one, sign in or reset your password", and that account's owner is notified. Residual risk: someone who controls a real Bangladeshi number can learn whether an e-mail address is registered. This is accepted, because each attempt costs a verified SIM and is rate-limited.
5. The e-mail link (`POST /auth/email/verify`) sets `email_verified_at`. It is required before the user's first listing, purchase or payout; this is checked by `@Verified`.

6.2 Sign-in, MFA and refresh

Figure 4 shows sign-in with MFA and the refresh rotation, including reuse detection.

Further rules:

- **Sign-in failure.** Unknown accounts, closed accounts and wrong passwords all receive the same `401` after the same Argon2id work, so response time does not reveal which accounts exist (SEC-AUTH-05). After `CP-04` failures, the delay before the next attempt doubles from 1 second up to 15 minutes. A successful password reset always clears the delay, so the owner is never locked out permanently (SEC-AUTH-04). The first delay notifies the owner.
- **TOTP.** Codes are accepted within one 30-second step either side of the current time. The step used is stored, so the same code cannot be used twice. A recovery code may replace the TOTP code once.

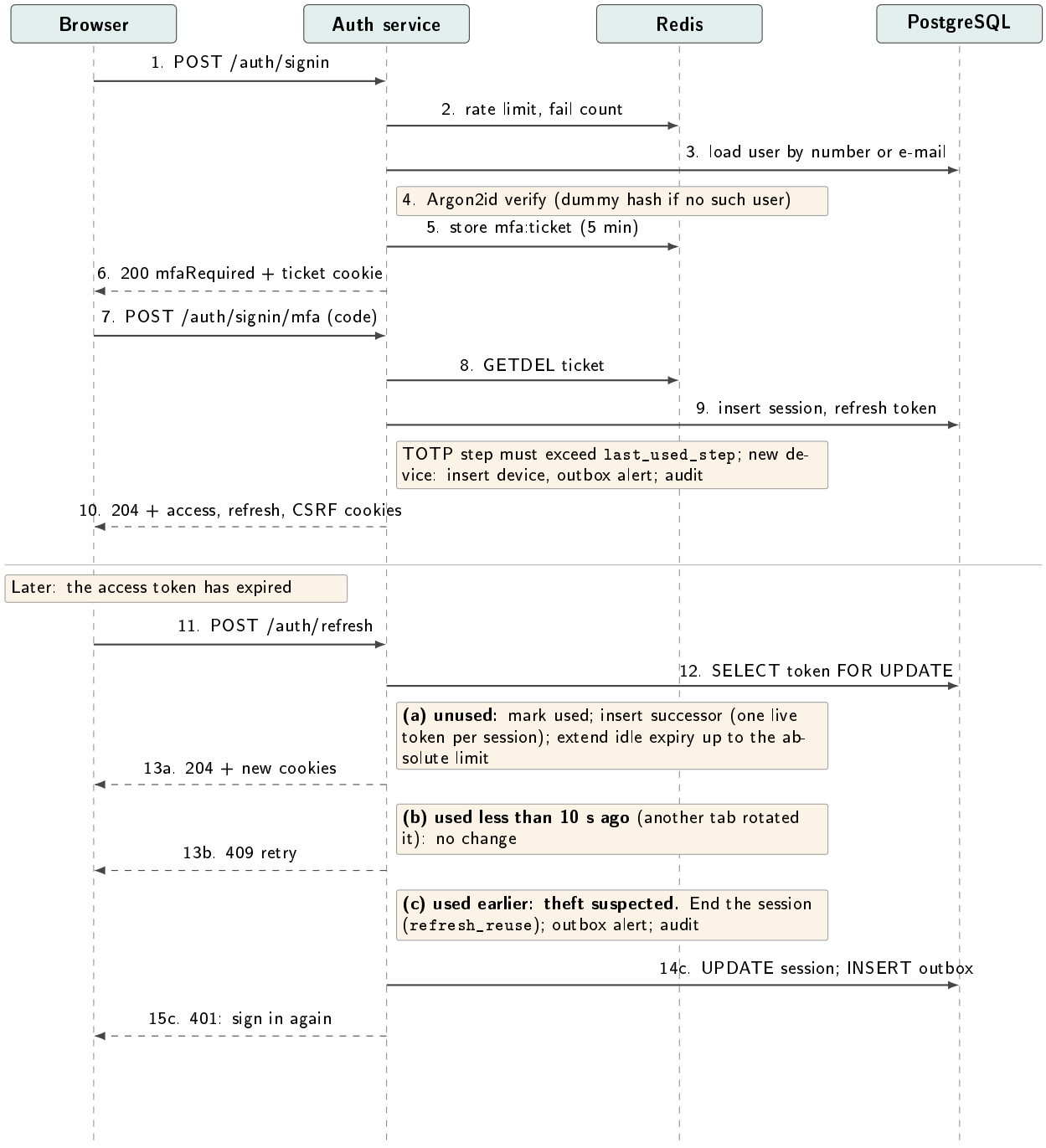


Figure 4: Sign-in with MFA, and refresh-token rotation with reuse detection (SEC-SES-02). Without MFA, steps 5–8 are skipped and the session is created after step 4.

- **Hash upgrades.** If the Argon2id parameters stored in a hash are older than the current ones, the password is re-hashed after a successful sign-in.
- **Parallel tabs.** The web client performs refreshes one at a time across tabs, using the browser's Web Locks API, so case (b) of Figure 4 is rare. The 10-second grace window (DV-4) covers the rest.
- **Session end.** Sign-out, "end session", password change, suspension, closure and staff disablement set `ended_at` and `end_reason`. The next request with any token of that session fails at AuthGuard. Open WebSocket connections in the room `sid:{id}` are disconnected through the Socket.IO adapter (SEC-SES-05, SEC-SES-07).

6.3 Step-up authentication

1. `POST /auth/step-up/start` names one action from a fixed list. The actions are: `password.change`, `phone.change`, `email.change`, `mfa.disable`, `recovery.regenerate`, `payout.change` (used by the Payment unit), `game.release_early` (used by the Game-Account unit) and `account.close`.
2. If the user has TOTP, no message is sent and the TOTP code is the challenge. Otherwise an OTP with purpose `step_up` is sent to the verified number, bound to the session id and the action.
3. `POST /auth/step-up/verify` checks the code and returns a single-use grant. Only its hash is kept in Redis, under `stepup:{sid}:{action}`, for DV-2.
4. The sensitive request carries the grant in the `X-Step-Up` header. StepUpGuard reads and deletes the key atomically (`GETDEL`) and compares the hash, so a grant works once, for one action, in one session (SEC-AUTH-08).

6.4 Password reset and change

- `POST /auth/password/forgot` always returns `202`. If the identifier is a registered e-mail, a 256-bit link token valid for CP-06 is e-mailed. If it is a registered number, a reset OTP is sent by SMS.
- `POST /auth/password/reset` takes the token (or number and OTP) and the new password. **If the account has MFA enabled, a TOTP or recovery code is also required**, so that access to the e-mail inbox alone cannot remove the second factor. A user who has lost both the authenticator and the recovery codes must re-verify their identity with an operator; that procedure belongs to the Staff Tools unit.
- On success, one transaction: sets the new hash and `password_single_factor_ok`; marks the token used; ends every session with reason `password_changed`; writes `identity.password.changed` to the outbox (SEC-AUTH-10, SEC-AUTH-11).
- `POST /auth/password/change` requires step-up and the current password, and ends every *other* session.

6.5 Contact changes and MFA

- **New number** (FR-ACC-09): step-up, then an OTP to the *new* number; on verification the number is replaced, but only if no other open account holds it (unique index). `identity.contact.changed` is sent to the *old* number.
- **New e-mail**: step-up, then a link token of purpose `email_change` sent to the new address; using it replaces the address and notifies the old one.
- **Enable TOTP**: a 160-bit secret is generated and stored encrypted but unconfirmed, and returned as an `otpauth://` URI shown as a QR code. Confirming with a valid code sets `confirmed_at` and `users.mfa_enabled`, and issues ten single-use recovery codes, shown once

and stored hashed.

- **Disable TOTP**: step-up plus a current TOTP code. If the password was set under the 8-character MFA rule (`password_single_factor_ok` is false), the same request must set a new password of at least 15 characters. The database rejects any other outcome (SEC-AUTH-01).

6.6 KYC submission

1. The user must be in `unsubmitted` or `rejected` status and within the resubmission limit CP-34.
2. `GET /kyc/liveness-challenge` returns a challenge id and one random action, such as "turn your head to the left". The wizard captures two frames through `getUserMedia`: one neutral, and one while the user performs the action. Gallery files are not accepted (FR-KYC-02).
3. `POST /kyc/submissions` (multipart): document type, number, full name, date of birth, the challenge id and four or five images. These are front and back, or passport page, plus the two selfie frames.
4. Each image is processed in order:
   (a) its type is identified from its content and must be JPEG, PNG or WebP of at most 10 MB;
   (b) it is decoded and rotated upright, all metadata including EXIF location is removed, it is resized to at most 2000 pixels on the long side, and it is re-encoded as JPEG;
   (c) it is hashed, encrypted with a fresh data key and written to the vault (Section 5.5).

   Re-encoding also destroys most payloads hidden in image files. A separate malware scanner would need a fifth container, which NFR-MNT-01 rules out, so SEC-INP-04's "should scan" is met by re-encoding, and this is recorded as a deviation in the ASVS checklist (SEC-GEN-01).
5. The service computes:
   - `is_adult` from the date of birth;
   - both blind indexes;
   - the routing flags: `underage`; `duplicate` (the document index is claimed by, or pending for, another user); `possible_duplicate` (the person index matches another user); `blacklisted`.

   Flags only route a submission for closer review; they never decide it (FR-KYC-03, FR-KYC-04).
6. One transaction:
   - inserts the submission and its file rows;
   - moves `kyc_status` to `pending_review`;
   - writes the history row;
   - audits;
   - writes `identity.kyc.submitted` to the outbox.

   The optional pre-check (FR-KYC-11) runs afterwards and may add the `precheck` flag and its advisory result.

The liveness check is basic, as the SRS states: a user who bypasses the web client can still upload prepared images. Its value is that it makes a reused photograph harder to pass and gives the operator two frames to compare. The operator's review remains the actual control.

6.7 KYC review, revocation and blacklist

1. **Queue.** The queue lists pending submissions oldest first, showing their flags and their age against CP-44 (FR-KYC-12).
2. **Claim.** An operator claims a submission with a single conditional update: it is assigned only if it is still pending and either unclaimed or its claim has expired (DV-6, 30 minutes). Two operators can therefore never work on the same submission. Claiming is refused if the

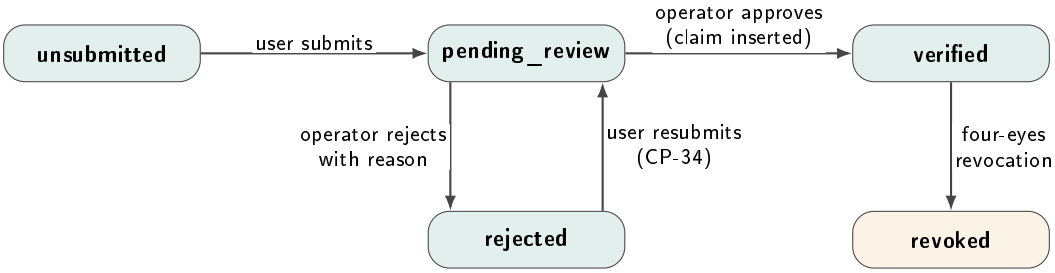


Figure 5: KYC status of a user (FR-KYC-05). The database trigger `users_kyc_transition` rejects every other change; `revoked` is final for that account.

operator has a conflict with the user: the same mobile number, e-mail or NID blind index as the operator's own, or a declared conflict (FR-OPS-06).

3. **Review.** Only the claimant can open the details. Document fields are decrypted for display, and every decryption is audited. Each image is opened through a signed link that requires a reason (Section 6.8).
4. **Approve.** One transaction:
   - locks the submission and re-checks the blacklist;
   - inserts the `identity_claims` row; if another account already holds the identity, the primary key rejects it, and the operator must reject the submission with `duplicate_identity`;
   - marks the submission approved and moves the user to `verified`;
   - writes the history row and the audit entry, and `identity.kyc.verified` to the outbox.
5. **Reject.** A reason code from the fixed list is required, with an optional note shown to the user. The user moves to `rejected`, and `identity.kyc.rejected` is written to the outbox (FR-KYC-06).
6. **Revoke** (FR-KYC-10). An operator proposes `kyc.revoke` with a reason. When a second operator approves, the handler moves the user to `revoked`, records the approval in the history and writes `identity.kyc.revoked` to the outbox, so that listings are hidden and new bids and purchases refused. The identity claim stays, so the same person cannot verify a new account.
7. **Blacklist** (FR-TNS-06, FR-GAM-18). `blacklist.add` is proposed with the user, the source and the related order. On approval, the handler copies the user's claimed blind index into `nid_blacklist`. Removing an entry needs its own approved proposal. Each new KYC submission is checked against active entries.

6.8 Protected files

A request to view an image (by the owner, or by the claiming operator with a reason) returns a link `/api/v1/files/v/{token}`. The token is the file id, the requesting session id and an expiry within CP-31, authenticated with HMAC under `LINK_KEY`. When the link is used, the server:

- verifies the HMAC and the expiry;
- checks that the request comes from the same session that asked for the link, so a leaked link is useless;
- decrypts the file as a stream;
- sends it with `Cache-Control: no-store`, `X-Content-Type-Options: nosniff`, `Content-Security-Policy: sandbox` and its fixed image type.

Issuing a link to staff is audited as `kyc.file.viewed` (SEC-ACC-06, SEC-DAT-04). The Disputes unit will issue links to case evidence through the same `FileVault` port, after its own check that the operator is assigned to the case.

6.9 Four-eyes approvals

`ApprovalService.propose()` writes an approval row with the action, target, payload, reason, required role and expiry (DV-5, 72 hours). Staff of the required role see open proposals in their queue. `decide()` records the decision. If it is "approved", the decision also calls the handler registered for that action *in the same transaction* and sets `executed_at`. Either the decision and its effect both happen, or neither does. The database prevents self-approval and duplicate open proposals, and a scheduled job marks stale proposals `expired`. Actions registered by this unit: `kyc.revoke`, `blacklist.add`, `blacklist.remove`, `account.reinstate` and `staff.mfa_reset`. Later units register theirs, for example large dispute outcomes (CP-30).

6.10 Suspension, closure and erasure

- **Suspension.** The Trust & Safety unit calls `IdentityCommands.suspendAccount`. This sets `account_status = suspended`, ends all sessions and writes `identity.account.suspended` to the outbox. The user can sign in again to follow paid orders, but `@Verified` refuses every new listing, bid and purchase (FR-TNS-05). Reinstatement is the four-eyes action `account.reinstate`.
- **Closure** (FR-ACC-11). Step-up, then every registered `AccountClosureCheck` is asked for blocking reasons. If any are returned, the user sees them. Otherwise one transaction:
  - sets `closed`, `closed_at` and `erase_after = closed_at + CP-41`;
  - ends all sessions;
  - writes `identity.account.closed` to the outbox.
- **Erasure** (FR-PRV-04). A daily job processes accounts past `erase_after` that no unit holds (the same checks, now asking about legal or dispute holds). It:
  - replaces the display name with "Deleted user" and empties the number, e-mail, password hash and location;
  - removes MFA factors;
  - deletes the identity claim;
  - erases KYC submissions: their encrypted fields and indexes are cleared, image files deleted and data keys destroyed;
  - sets `erased_at`.

  The user row itself remains, because orders and settlements in other units reference it. A blacklist entry keeps its own blind index (SRS Table 7.2). The same job erases KYC data of submissions rejected more than 30 days earlier when the user has not resubmitted.

6.11 Staff accounts

- The bootstrap command creates the first administrator in status `pending_mfa`. An administrator creates further staff accounts the same way. The initial password is generated, shown once to the creating administrator, and handed over in person.
- A staff member in `pending_mfa` can only change the password and enrol TOTP or a passkey; then the status becomes `active` (SEC-AUTH-07).
- Disabling a staff account ends its sessions. Resetting a staff member's MFA is the four-eyes action `staff.mfa_reset`, which needs a second administrator. With a team of one administrator, it falls back to the bootstrap procedure, which is audited.
- Staff sessions use CP-03 limits. Staff accounts cannot reach any `@Verified` endpoint, because a staff principal is never a user principal (FR-OPS-04).

7 Security Design

7.1 Tokens and cookies

Cookie	Content	Attributes	Lifetime
`__Host-tb_at`	Access JWT (HS256, `kid`; claims `sub`, `sid`, `pt`, `iat`, `exp`)	HttpOnly, Secure, SameSite=Strict, Path=/	CP-01 (15 min)
`__Secure-tb_rt`	256-bit random refresh token (server keeps SHA-256)	HttpOnly, Secure, SameSite=Strict, Path=/api/v1/auth/refresh	Session's absolute limit
`__Host-tb_csrf`	HMAC of the session id (signed double-submit)	Secure, SameSite=Strict, Path=/; *readable* by the client, which copies it into `X-CSRF-Token`	Same as access token
`__Host-tb_mfa`	MFA ticket (server keeps hash)	HttpOnly, Secure, SameSite=Strict, Path=/	DV-1 (5 min)
`__Host-tb_dev`	256-bit device key (server keeps hash)	HttpOnly, Secure, SameSite=Strict, Path=/	1 year

No token is ever placed in `localStorage`, a URL or a log (SEC-SES-03). The `__Host-` prefix makes the browser refuse a cookie that is not Secure, has a Domain or has a path other than `/`, which blocks cookie injection from sibling sub-domains. The refresh cookie uses `__Secure-` because it is limited to the refresh path, so the refresh token is sent with no other request.

Same origin in development and production. The web client and the API are served from one origin: Vite's development proxy forwards `/api` to the backend, and in production a reverse proxy does the same. All cookies are therefore first-party, SameSite=Strict works, and CORS is needed only for listed tools (SEC-WEB-03). Browsers treat `http://localhost` as a secure context, so Secure cookies work in local development; Safari needs the development server's HTTPS option.

Changes needed in the repository. `frontend/src/api/client.ts` must stop reading a token from `localStorage`, send requests with `credentials: 'include'`, and add the `X-CSRF-Token` header. `bcrypt` is replaced by `argon2`. The current `User` and `KycDocument` entities in `backend/src/entities` are replaced by entities matching this schema; their status values (`unverified`, `pending`, `approved`) do not match the SRS, and the single `role` column mixes Pro Seller with staff roles, which this design separates.

7.2 Threats and how the design answers them

Threat	Design response
Credential stuffing	Per-account progressive delay and per-address limits; breached-password blocklist; MFA option; new-device alerts (SEC-AUTH-02, SEC-AUTH-04, SEC-AUTH-12).
Account enumeration	Identical responses and equal Argon2id work for unknown accounts; registration does not reveal an existing number; reset always returns `202` (SEC-AUTH-05, SEC-AUTH-06). Residual: e-mail existence can be learned by someone who verifies a real SIM, at a rate-limited cost (Section 6.1).
SMS pumping (fraudulent OTP traffic)	Only `+8801` mobile numbers are accepted, so premium international numbers are impossible; per-number and per-address send limits; no OTP is sent for an already-registered number.

Threat	Design response
Session theft	HttpOnly cookies; short access tokens; refresh rotation with reuse detection that ends the session; server-side session check on every request; "end other sessions" (SEC-SES-02, SEC-SES-05).
CSRF	SameSite=Strict, origin check and signed double-submit token (SEC-SES-04).
Account takeover via e-mail	Password reset requires the second factor when MFA is on; contact changes need step-up and notify the old contact.
IDOR on documents	Signed links bound to the requesting session; claim-based access for operators; object checks on every route (SEC-ACC-02, SEC-ACC-06).
Insider browsing KYC	Claim required; reason required per image; every view and decryption audited; alert on volume (SEC-LOG-05); administrators cannot see KYC at all (FR-OPS-03).
Database or backup theft	Field and file encryption with keys outside the database; HMAC blind indexes; OTP pepper; hashed tokens and codes (SEC-DAT-02, SEC-DAT-03, SEC-DAT-06).
Duplicate or banned identities	`identity_claims` primary key; person index as a second signal; blacklist check on every submission (FR-KYC-04, FR-TNS-06).
Log tampering	Append-only triggers and grants; hash chain with scheduled verification (SEC-LOG-02).

8 Parameters

8.1 SRS parameters used

CP-01, CP-02 and CP-03 (session limits); CP-04 (sign-in failures); CP-05 (OTP); CP-06 (link tokens); CP-30 (four-eyes actions); CP-31 (signed-link lifetime); CP-33 (upload limits); CP-34 (KYC resubmission); CP-35 (public profile reads); CP-41 (erasure after closure); CP-44 (KYC review target).

8.2 Design values introduced by this unit

The values below are needed by the design but are not yet in SRS Table 8.1. They are proposed for addition to the SRS as CP-48 onwards when the integration step is done.

ID	Value	Default	Reason
DV-1	MFA ticket lifetime	5 minutes	Time to open the authenticator app.
DV-2	Step-up grant lifetime	5 minutes	One sensitive action, completed promptly.
DV-3	Liveness challenge lifetime	10 minutes	Capture and upload on a slow connection.
DV-4	Refresh reuse grace window	10 seconds	Parallel tabs; beyond it, reuse means theft.
DV-5	Approval proposal expiry	72 hours	A stale proposal must be re-proposed.
DV-6	KYC claim duration	30 minutes	Released if the operator leaves.
DV-7	Recovery codes issued	10	Common practice.
DV-8	Maximum password length accepted	256 characters	SRS requires at least 64; bounds Argon2id work.
DV-9	Stored KYC image size	2000 px long side	Legible for review; bounded storage.

ID	Value	Default	Reason
DV-10	Sign-in delay after CP-04 failures	1 s doubling to 15 min	Slows guessing without permanent lock-out.

9 Testing

Level	Cases
Database (done)	46 checks in `01_identity_access_tests.sql` (Appendix B.1), plus the concurrent audit-chain test (8 writers, 480 entries, chain intact).
Unit	Password policy, including a 15-character Bangla passphrase and blocklist hits; Argon2id parameters and re-hash; OTP HMAC and constant-time comparison; TOTP window and replay refusal; field encryption fails when a ciphertext is moved to another row (associated data); blind-index normalisation; CSRF token binding; signed-link expiry and session binding.
Integration (PostgreSQL + Redis)	Registration with a new number, an existing number and an existing e-mail give identical first responses. 20 parallel refreshes of one token produce exactly one rotation. Reuse after the grace window ends the session. An ended session is refused on its next request and its WebSocket is disconnected. Two parallel approvals of the same NID for two accounts: exactly one succeeds. Two operators claiming one submission: exactly one succeeds. Self-approval is refused. A file link used from another session is refused. An uploaded photo with GPS EXIF data is stored without it. Rate limits hold across two backend instances.
Security (SEC-SDL-09)	For every endpoint in Section 4.2: no session, an expired session, another user's object, and a lower role, each refused with the correct status.
Acceptance	The KYC gate: a user in each non-verified status is refused by `@Verified` for list, bid, buy and payout (FR-KYC-07).

10 Notes for Integration

These points are recorded now so that they are settled when the units are connected.

I-1 User location. The saved approximate location is stored here. The Listings unit decides whether each listing copies it (FR-LST-06).

I-2 Pro Seller status belongs to the Monetization part of the Order unit; `IdentityQuery` does not expose it.

I-3 Message templates and their Bangla and English texts belong to the Notifications unit; this unit supplies event data only.

I-4 Parameter store. `ConfigPort` moves from environment configuration to the Staff Tools unit's versioned store (FR-OPS-08) without changing callers.

I-5 Lost MFA and recovery codes. An operator-assisted re-verification procedure is to be designed in the Staff Tools unit.

I-6 SRS updates proposed:

- add DV-1 to DV-10 to the SRS parameter table;
- state in FR-KYC-02 that two selfie frames are captured;
- state in FR-ACC-07 that a reset requires the second factor when MFA is enabled;
- record the SEC-INP-04 malware-scan deviation in the ASVS checklist.

I-7 Shared infrastructure first. The `security` and `audit` modules, and `ApprovalService`, should be implemented first, because every later unit imports them.

A Traceability

Requirement	Design element	Section
FR-ACC-01	Registration endpoint; pending_registrations; phone parsing	6.1
FR-ACC-02	Phone OTP before users row exists; e-mail link token; `@Verified` requires e-mail	6.1
FR-ACC-03	Partial unique indexes on number and e-mail; enumeration-safe responses	5.2, 6.1
FR-ACC-04	Age confirmation; consents rows with versions	6.1
FR-ACC-05	Sign-in flow; session creation	6.2
FR-ACC-06	totp_factors, recovery_codes; MFA ticket; enrol/disable flows	6.2, 6.5
FR-ACC-07	Forgot/reset endpoints; reset requires second factor when enabled	6.4
FR-ACC-08	Sessions list/end endpoints; known_devices	4.2, 6.2
FR-ACC-09	Step-up; OTP or link to new contact; notice to old contact	6.5
FR-ACC-10	`getPublicProfile` returns only permitted fields	4.3
FR-ACC-11	`AccountClosureCheck` port; closure flow	4.5, 6.10
FR-KYC-01	Submission endpoint; kyc_submissions, kyc_files	6.6
FR-KYC-02	Liveness challenge; two camera frames; no gallery upload	6.6
FR-KYC-03	`is_adult`; underage flag; DB forbids approving underage	5.2, 6.6
FR-KYC-04	identity_claims primary key; blind index; duplicate flags	5.3, 6.7
FR-KYC-05	kyc_status enum; transition trigger; kyc_status_history	Fig. 5
FR-KYC-06	kyc_reason enum; DB requires reason on rejection	6.7
FR-KYC-07	`@Verified` policy; `assertCanTransact` port	4.1, 4.3
FR-KYC-08	`kyc_status` exposed through `IdentityQuery`; events update listings	4.3, 4.4
FR-KYC-09	Resubmission from `rejected`; CP-34 count	6.6
FR-KYC-10	`kyc.revoke` four-eyes action; `identity.kyc.revoked` event	6.7
FR-KYC-11	IdentityCheckPort; advisory `precheck`; never decides	4.5, 6.6
FR-KYC-12	Queue ordered by age, shown against CP-44	6.7
FR-TNS-06	nid_blacklist; checked at submission and approval	6.7
FR-OPS-03	Administrator endpoints limited to staff, approvals, audit	4.2
FR-OPS-04	staff_accounts separate from users; bootstrap command; single bootstrap row	4.2, 6.11
FR-OPS-05	approvals table; proposer ≠ decider; handler in same transaction	6.9
FR-OPS-06	Conflict check at claim (number, e-mail, NID, declared)	6.7
FR-OPS-07	AuditService in every staff action	DC-8
FR-PRV-01	Consent and notice versions recorded at registration	6.1
FR-PRV-02	Optional consents endpoint	4.2
FR-PRV-04	Erasure job; anonymisation; crypto-shredding	6.10
FR-PRV-05	KYC data reachable only by review flows and `getKycLegalName`	4.3, 6.7
FR-NTF-06	Security events through the outbox	4.4
SEC-AUTH-01	15/8 rule; `password_single_factor_ok`; DB check	6.5
SEC-AUTH-02	Bundled blocklist and context words	6.1
SEC-AUTH-03	Argon2id only (DB check); re-hash on sign-in	6.2
SEC-AUTH-04	signin rule; progressive delay (DV-10)	5.4, 6.2
SEC-AUTH-05	Dummy hash; identical responses	6.2
SEC-AUTH-06	Registration notice to owner; same response	6.1
SEC-AUTH-07	Staff MFA mandatory; `pending_mfa`; no SMS factor	6.11
SEC-AUTH-08	StepUpService; single-use grant bound to session and action	6.3
SEC-AUTH-09	otp_challenges with HMAC, attempts cap, send limits	5.2, 5.4
SEC-AUTH-10	256-bit link tokens, hashed, single-use; sessions ended	6.4
SEC-AUTH-11	Password change ends other sessions	6.4
SEC-AUTH-12	known_devices; device cookie	6.2
SEC-SES-01	HS256 only, `kid`, 15-min access token	7.1
SEC-SES-02	refresh_tokens rotation; one live token; reuse detection	Fig. 4
SEC-SES-03	HttpOnly cookies; no local storage	7.1
SEC-SES-04	Signed double-submit CSRF and origin check	4.1

Requirement	Design element	Section
SEC-SES-05	Session row checked on every request	4.1
SEC-SES-06	CP-03 limits for staff sessions	6.11
SEC-SES-07	WebSocket handshake authentication; `sid` rooms	6.2
SEC-ACC-01	Global deny-by-default guard (DC-7)	4.1
SEC-ACC-02	Object checks; claim-based access; signed links	6.7, 6.8
SEC-ACC-03	Policy decorators per role	4.1
SEC-ACC-04	Whitelisted DTOs (DC-3)	2
SEC-ACC-05	Masked staff views; full value only during review, audited	6.7
SEC-ACC-06	Claim, reason and audit for every KYC image view	6.7, 6.8
SEC-DAT-01	TLS and HSTS (DC-11)	2
SEC-DAT-02	AES-256-GCM field and file encryption; keys outside the database	5.5
SEC-DAT-03	Encrypted document number; HMAC blind indexes	5.5
SEC-DAT-04	Signed links bound to session, CP-31	6.8
SEC-DAT-05	Versioned secrets from environment; start-up check	5.5
SEC-DAT-06	Crypto-shredding; backup note	5.5
SEC-DAT-07	Masked contacts; no secrets in logs	5.5
SEC-DAT-08	Images re-encoded without metadata	6.6
SEC-DAT-09	Node.js `crypto` random bytes for every secret	3.3
SEC-DAT-11	Random UUID keys (DC-4)	2
SEC-INP-04	Content-type check, size limit, re-encoding, random names	6.6
SEC-ABU-01	Rate-limit rules table	5.4
SEC-ABU-03	Counters in Redis shared by all instances	5.4
SEC-LOG-01	AuditService; event list	DC-8
SEC-LOG-02	Hash chain trigger; `verify_chain()`	5.3
SEC-LOG-03	`clock_timestamp()` under lock; correlation id	5.3
SEC-LOG-04	Grants: insert and select only; admin read-only view	5.3
SEC-LOG-05	Volume alert on image links	5.4
IF-UI-01	Registration, sign-in, MFA, security settings views	3.2
IF-UI-02	KYC wizard and status	6.6
IF-UI-09	Staff console views	4.2
IF-SW-05	NotificationPort (SMS)	4.5
IF-SW-06	NotificationPort (e-mail)	4.5
NFR-USE-02	Status endpoint with reason; wizard states	4.2
DR-01	users, consents, sessions, factors	5
DR-02	kyc_submissions, kyc_files, history	5
DR-09	staff_accounts, approvals, audit.events	5
DR-10	Append-only triggers and grants	5.3

B Database Schema and Tests

B.1 Test results

The schema was loaded into PostgreSQL 16 (only features available in PostgreSQL 15, the project's target, are used) and the test script was run. Every case passed:

```
PASS  only one bootstrap administrator (rejected: duplicate key value violates unique constraint
        "staff_single_bootstrap)
PASS  staff password must be Argon2id (rejected: new row for relation "staff_accounts" violates
        check constraint "staff)
PASS  FR-ACC-03 duplicate live mobile number (rejected: duplicate key value violates unique
        constraint "users_phone_live")
PASS  FR-ACC-03 duplicate live e-mail, case-insensitive (rejected: duplicate key value violates
        unique constraint "users_email_live")
PASS  Bangladeshi mobile format enforced (rejected: new row for relation "users" violates check
        constraint "users_phone_e1)
PASS  SEC-AUTH-01 short password allowed only with MFA (rejected: new row for relation "users"
        violates check constraint "users_check3")
PASS  SEC-AUTH-01 cannot switch MFA off while password is short (rejected: new row for relation
```

```
        "users" violates check constraint "users_check3")
PASS  personal fields removable only after erasure (rejected: new row for relation "users" violates
        check constraint "users_check")
PASS  erased account frees its number and e-mail
PASS  FR-KYC-05 unsubmitted -> verified not permitted (rejected: kyc_status unsubmitted -> verified
        is not permitted)
PASS  unsubmitted -> pending_review -> verified permitted
PASS  FR-KYC-05 verified -> pending_review not permitted (rejected: kyc_status verified ->
        pending_review is not permitted)
PASS  one open KYC submission per user (rejected: duplicate key value violates unique constraint
        "kyc_one_pending")
PASS  FR-KYC-06 rejection needs a reason code (rejected: new row for relation "kyc_submissions"
        violates check constraint "kyc_")
PASS  FR-KYC-03 underage submission cannot be approved (rejected: new row for relation
        "kyc_submissions" violates check constraint "kyc_")
PASS  FR-KYC-04 same NID cannot verify a second account (rejected: duplicate key value violates
        unique constraint "identity_claims_pkey")
PASS  routed flags restricted to known values (rejected: new row for relation "kyc_submissions"
        violates check constraint "kyc_")
PASS  SEC-INP-04 only JPEG/PNG/WebP accepted (rejected: new row for relation "kyc_files" violates
        check constraint "kyc_files_")
PASS  CP-33 10 MB upload limit (rejected: new row for relation "kyc_files" violates check constraint
        "kyc_files_")
PASS  erased file must have its key destroyed (rejected: new row for relation "kyc_files" violates
        check constraint "kyc_files_")
PASS  KYC history is append-only (rejected: table identity.kyc_status_history is append-only)
PASS  KYC history rows cannot be deleted (rejected: table identity.kyc_status_history is append-
        only)
PASS  FR-OPS-05 proposer cannot approve own action (rejected: new row for relation "approvals"
        violates check constraint "approvals_")
PASS  one open proposal per action and target (rejected: duplicate key value violates unique
        constraint "approvals_one_open")
PASS  decider recorded without a decision (rejected: new row for relation "approvals" violates check
        constraint "approvals_")
PASS  cannot execute before approval (rejected: new row for relation "approvals" violates check
        constraint "approvals_")
PASS  one active blacklist entry per identity (rejected: duplicate key value violates unique
        constraint "blacklist_active")
PASS  blacklist removal needs an approval (rejected: new row for relation "nid_blacklist" violates
        check constraint "nid_bl")
PASS  session belongs to exactly one principal type (rejected: new row for relation "sessions"
        violates check constraint "sessions_ch")
PASS  idle expiry cannot exceed absolute expiry (rejected: new row for relation "sessions" violates
        check constraint "sessions_ch")
PASS  SEC-SES-02 only one live refresh token per session (rejected: duplicate key value violates
        unique constraint "refresh_one_live")
PASS  rotation leaves exactly one live token
PASS  a reused token is detectable (used_at set)
PASS  SEC-AUTH-09 attempts cannot exceed the limit (rejected: new row for relation "otp_challenges"
        violates check constraint "otp_c")
PASS  register OTP must belong to a pending registration (rejected: new row for relation
        "otp_challenges" violates check constraint "otp_c")
PASS  SEC-LOG-02 chain verifies when intact
PASS  each entry links to the previous hash
PASS  audit entries cannot be updated (rejected: table audit.events is append-only)
PASS  audit entries cannot be deleted (rejected: table audit.events is append-only)
PASS  action names follow the dotted convention (rejected: new row for relation "events" violates
        check constraint "events_action")
PASS  SEC-LOG-02 verification finds the tampered entry
PASS  outbox event names belong to this unit (rejected: new row for relation "outbox" violates check
        constraint "outbox_event_")
PASS  SEC-INF-03 application role cannot change the schema (rejected: must be owner of table users)
PASS  SEC-LOG-04 application role cannot update audit entries (rejected: permission denied for table
        events)
PASS  application role cannot delete users (rejected: permission denied for table users)
PASS  application role can append to the audit log
```

B.2 Schema

```
-- ================================================================
-- TrustBazaar  -  Unit 1: Identity & Access  -  PostgreSQL 15 schema
-- Design document: docs/design/01_identity_access.pdf
-- Requirements: SRS v2.0 (FR-ACC, FR-KYC, FR-OPS-03..07, FR-PRV-01,
```

```
--               FR-TNS-06, SEC-AUTH, SEC-SES, SEC-ACC, SEC-DAT, SEC-LOG)
--
-- Conventions (design convention DC-4):
--   * one schema per unit; this unit owns schemas "identity" and "audit"
--   * primary keys are random UUIDs (gen_random_uuid), never sequential
--     for objects that appear in URLs (SEC-DAT-11)
--   * all times are timestamptz (UTC)
--   * *_enc columns hold AES-256-GCM ciphertext produced by the
--     application ("v<key>|iv|ciphertext|tag"); *_bidx columns hold
--     HMAC-SHA-256 blind indexes (SEC-DAT-03); *_hash columns hold
--     SHA-256 of a random secret, or an Argon2id string for passwords
--   * append-only tables are protected by trigger AND by grants
-- Must run as the migration role (tb_migrator); the application connects
-- as tb_app, which cannot change the schema (SEC-INF-03).
-- ====================================================================

CREATE EXTENSION IF NOT EXISTS citext;
CREATE EXTENSION IF NOT EXISTS pgcrypto;   -- digest() for the audit hash chain

CREATE SCHEMA IF NOT EXISTS identity;
CREATE SCHEMA IF NOT EXISTS audit;

-- ------------------------------------------------------------------ types
CREATE TYPE identity.kyc_status AS ENUM
  ('unsubmitted', 'pending_review', 'verified', 'rejected', 'revoked');          -- FR-KYC-05
CREATE TYPE identity.account_status AS ENUM ('active', 'suspended', 'closed');
CREATE TYPE identity.kyc_submission_status AS ENUM
  ('pending_review', 'approved', 'rejected');
CREATE TYPE identity.kyc_doc_type AS ENUM ('nid', 'passport');
CREATE TYPE identity.kyc_file_kind AS ENUM
  ('doc_front', 'doc_back', 'passport_photo_page', 'selfie_neutral', 'selfie_action');
CREATE TYPE identity.kyc_reason AS ENUM                                         -- FR-KYC-06
  ('image_unreadable', 'selfie_mismatch', 'liveness_failed', 'document_expired',
   'details_mismatch', 'underage', 'duplicate_identity', 'blacklisted', 'other');
CREATE TYPE identity.principal_type AS ENUM ('user', 'staff');
CREATE TYPE identity.staff_role AS ENUM ('operator', 'administrator');
CREATE TYPE identity.staff_status AS ENUM ('pending_mfa', 'active', 'disabled');
CREATE TYPE identity.otp_purpose AS ENUM
  ('register_phone', 'step_up', 'contact_change_phone', 'password_reset');
CREATE TYPE identity.token_purpose AS ENUM ('email_verify', 'email_change', 'password_reset');
CREATE TYPE identity.session_end_reason AS ENUM
  ('logout', 'user_ended', 'password_changed', 'refresh_reuse', 'suspended',
   'closed', 'staff_disabled', 'emergency', 'mfa_reset');
CREATE TYPE identity.approval_decision AS ENUM ('approved', 'rejected', 'expired');
CREATE TYPE audit.actor_type AS ENUM ('user', 'staff', 'system', 'provider');

-- ------------------------------------------------------------------ shared trigger: forbid UPDATE/DELETE
CREATE FUNCTION identity.forbid_change() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  RAISE EXCEPTION 'table␣%.%␣is␣append-only', TG_TABLE_SCHEMA, TG_TABLE_NAME
    USING ERRCODE = 'insufficient_privilege';
END $$;

-- ====================================================================
-- Staff accounts (FR-OPS-03, FR-OPS-04, SEC-AUTH-07)
-- Kept separate from users so that a staff account can never buy, sell or bid.
-- ====================================================================
CREATE TABLE identity.staff_accounts (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email           citext NOT NULL UNIQUE CHECK (length(email) <= 254),
  display_name    varchar(60) NOT NULL,
  role            identity.staff_role NOT NULL,
  status          identity.staff_status NOT NULL DEFAULT 'pending_mfa',
  password_hash   text NOT NULL CHECK (password_hash LIKE '$argon2id$%'),
  -- used for the conflict-of-interest check of FR-OPS-06
  phone_e164      varchar(16) CHECK (phone_e164 ~ '^\+8801[3-9][0-9]{8}$'),
  nid_bidx        bytea CHECK (octet_length(nid_bidx) = 32),
  created_by      uuid REFERENCES identity.staff_accounts(id),   -- NULL only for the first administrator
  created_at      timestamptz NOT NULL DEFAULT now(),
  disabled_at     timestamptz,
  disabled_by     uuid REFERENCES identity.staff_accounts(id),
  CHECK ((status = 'disabled') = (disabled_at IS NOT NULL))
);
-- only one administrator may exist without a creator (the bootstrap one)
CREATE UNIQUE INDEX staff_single_bootstrap ON identity.staff_accounts ((true)) WHERE created_by IS NULL;
```

```
CREATE TABLE identity.staff_conflicts (                                    -- FR-OPS-06
  staff_id     uuid NOT NULL REFERENCES identity.staff_accounts(id),
  user_id      uuid NOT NULL,          -- FK added after identity.users exists
  reason       text NOT NULL,
  declared_at  timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (staff_id, user_id)
);

-- ================================================================
-- Users (FR-ACC-01..11, FR-KYC-05)
-- ================================================================
CREATE TABLE identity.users (
  id                        uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  display_name              varchar(40) NOT NULL CHECK (length(btrim(display_name)) >= 2),
  phone_e164                varchar(16) CHECK (phone_e164 ~ '^\+8801[3-9][0-9]{8}$'),
  phone_verified_at         timestamptz NOT NULL,          -- account exists only after OTP (FR-ACC-02)
  email                     citext CHECK (length(email) <= 254 AND email LIKE '%_@_%'),
  email_verified_at         timestamptz,
  password_hash             text CHECK (password_hash LIKE '$argon2id$%'),   -- SEC-AUTH-03
  password_single_factor_ok boolean NOT NULL,              -- length >= 15 when set (SEC-AUTH-01)
  password_changed_at       timestamptz NOT NULL DEFAULT now(),
  mfa_enabled               boolean NOT NULL DEFAULT false,
  kyc_status                identity.kyc_status NOT NULL DEFAULT 'unsubmitted',
  kyc_status_changed_at     timestamptz NOT NULL DEFAULT now(),
  account_status            identity.account_status NOT NULL DEFAULT 'active',
  area_district             varchar(40),
  approx_lat                numeric(6,3) CHECK (approx_lat BETWEEN 20.0 AND 27.0),  -- ~110 m grid,
    Bangladesh
  approx_lng                numeric(6,3) CHECK (approx_lng BETWEEN 88.0 AND 93.0),
  adult_confirmed_at        timestamptz NOT NULL,          -- FR-ACC-04
  created_at                timestamptz NOT NULL DEFAULT now(),
  updated_at                timestamptz NOT NULL DEFAULT now(),
  closed_at                 timestamptz,
  erase_after               timestamptz,                   -- closed_at + CP-41
  erased_at                 timestamptz,                   -- personal fields removed (FR-PRV-04)
  -- contact details and password may be absent only after erasure
  CHECK (erased_at IS NOT NULL OR (phone_e164 IS NOT NULL AND email IS NOT NULL AND password_hash IS NOT NULL
    )),
  CHECK (erased_at IS NULL OR account_status = 'closed'),
  CHECK ((account_status = 'closed') = (closed_at IS NOT NULL)),
  -- a password shorter than 15 characters is allowed only while MFA is on (SEC-AUTH-01)
  CHECK (password_single_factor_ok OR mfa_enabled),
  CHECK ((approx_lat IS NULL) = (approx_lng IS NULL))
);
-- contact details are unique among accounts that are not closed (FR-ACC-03)
CREATE UNIQUE INDEX users_phone_live ON identity.users (phone_e164) WHERE account_status <> 'closed';
CREATE UNIQUE INDEX users_email_live ON identity.users (email)      WHERE account_status <> 'closed';
CREATE INDEX users_erase_due ON identity.users (erase_after) WHERE erase_after IS NOT NULL;

ALTER TABLE identity.staff_conflicts
  ADD FOREIGN KEY (user_id) REFERENCES identity.users(id);

CREATE TABLE identity.consents (                                           -- FR-ACC-04, FR-PRV-01/02
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id      uuid NOT NULL REFERENCES identity.users(id),
  document     varchar(20) NOT NULL CHECK (document IN ('terms', 'privacy', 'marketing', 'recommendations')),
  version      varchar(20) NOT NULL,
  granted      boolean NOT NULL,
  recorded_at  timestamptz NOT NULL DEFAULT now(),
  source_ip    inet
);
CREATE INDEX consents_user ON identity.consents (user_id, document, recorded_at DESC);

-- Registration before the phone is verified. Nothing is created in "users"
-- until the OTP succeeds (FR-ACC-02); rows expire with the OTP.
CREATE TABLE identity.pending_registrations (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  phone_e164       varchar(16) NOT NULL CHECK (phone_e164 ~ '^\+8801[3-9][0-9]{8}$'),
  email            citext NOT NULL,
  display_name     varchar(40) NOT NULL,
  password_hash    text NOT NULL CHECK (password_hash LIKE '$argon2id$%'),
  password_single_factor_ok boolean NOT NULL CHECK (password_single_factor_ok),  -- no MFA yet => >= 15 chars
  terms_version    varchar(20) NOT NULL,
  privacy_version  varchar(20) NOT NULL,
```

```
  adult_confirmed  boolean NOT NULL CHECK (adult_confirmed),
  created_at       timestamptz NOT NULL DEFAULT now(),
  expires_at       timestamptz NOT NULL,
  completed_at     timestamptz,
  source_ip        inet
);
CREATE INDEX pending_reg_phone ON identity.pending_registrations (phone_e164, created_at DESC);

-- ====================================================================
-- One-time secrets: OTPs (SEC-AUTH-09) and link tokens (SEC-AUTH-10)
-- Only hashes are stored.
-- ====================================================================
CREATE TABLE identity.otp_challenges (
  id                      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  purpose                 identity.otp_purpose NOT NULL,
  user_id                 uuid REFERENCES identity.users(id),
  pending_registration_id  uuid REFERENCES identity.pending_registrations(id),
  phone_e164              varchar(16) NOT NULL,
  code_hash               bytea NOT NULL CHECK (octet_length(code_hash) = 32),  -- HMAC-SHA-256(code, pepper
    )
  bound_to                text,         -- e.g. session id + action for step-up
  attempts                smallint NOT NULL DEFAULT 0 CHECK (attempts >= 0),
  max_attempts            smallint NOT NULL,                                    -- CP-05
  created_at              timestamptz NOT NULL DEFAULT now(),
  expires_at              timestamptz NOT NULL,
  consumed_at             timestamptz,
  CHECK (attempts <= max_attempts),
  CHECK (expires_at > created_at),
  CHECK ((purpose = 'register_phone') = (pending_registration_id IS NOT NULL)),
  CHECK (purpose = 'register_phone' OR user_id IS NOT NULL)
);
CREATE INDEX otp_phone_recent ON identity.otp_challenges (phone_e164, created_at DESC);

CREATE TABLE identity.one_time_tokens (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  purpose     identity.token_purpose NOT NULL,
  user_id     uuid NOT NULL REFERENCES identity.users(id),
  token_hash  bytea NOT NULL UNIQUE CHECK (octet_length(token_hash) = 32),    -- SHA-256 of 256-bit token
  new_email   citext,                                                         -- for email_change only
  created_at  timestamptz NOT NULL DEFAULT now(),
  expires_at  timestamptz NOT NULL,
  used_at     timestamptz,
  CHECK ((purpose = 'email_change') = (new_email IS NOT NULL))
);

-- ====================================================================
-- Second factors (FR-ACC-06, SEC-AUTH-07)
-- ====================================================================
CREATE TABLE identity.totp_factors (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  principal_type  identity.principal_type NOT NULL,
  user_id         uuid REFERENCES identity.users(id),
  staff_id        uuid REFERENCES identity.staff_accounts(id),
  secret_enc      bytea NOT NULL,          -- AES-256-GCM under the field key
  key_version     smallint NOT NULL,
  confirmed_at    timestamptz,             -- factor counts only once confirmed with a code
  last_used_step  bigint,                  -- rejects reuse of the same 30-s code
  created_at      timestamptz NOT NULL DEFAULT now(),
  removed_at      timestamptz,
  CHECK ((principal_type = 'user')  = (user_id  IS NOT NULL)),
  CHECK ((principal_type = 'staff') = (staff_id IS NOT NULL))
);
CREATE UNIQUE INDEX totp_one_active_user  ON identity.totp_factors (user_id)  WHERE removed_at IS NULL AND
    user_id  IS NOT NULL;
CREATE UNIQUE INDEX totp_one_active_staff ON identity.totp_factors (staff_id) WHERE removed_at IS NULL AND
    staff_id IS NOT NULL;

CREATE TABLE identity.recovery_codes (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  totp_factor_id  uuid NOT NULL REFERENCES identity.totp_factors(id),
  code_hash       bytea NOT NULL CHECK (octet_length(code_hash) = 32),
  used_at         timestamptz
);

CREATE TABLE identity.webauthn_credentials (                                  -- staff passkeys (optional)
```

```
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  staff_id         uuid NOT NULL REFERENCES identity.staff_accounts(id),
  credential_id    bytea NOT NULL UNIQUE,
  public_key       bytea NOT NULL,
  sign_count       bigint NOT NULL DEFAULT 0,
  created_at       timestamptz NOT NULL DEFAULT now(),
  last_used_at     timestamptz,
  removed_at       timestamptz
);

-- ====================================================================
-- Devices and sessions (FR-ACC-08, SEC-AUTH-12, SEC-SES-01..07)
-- A session is one sign-in on one device; its refresh tokens rotate.
-- ====================================================================
CREATE TABLE identity.known_devices (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         uuid NOT NULL REFERENCES identity.users(id),
  device_key_hash bytea NOT NULL CHECK (octet_length(device_key_hash) = 32),
  label           varchar(80),            -- e.g. "Chrome on Android"
  first_seen_at   timestamptz NOT NULL DEFAULT now(),
  last_seen_at    timestamptz NOT NULL DEFAULT now(),
  UNIQUE (user_id, device_key_hash)
);

CREATE TABLE identity.sessions (
  id                   uuid PRIMARY KEY DEFAULT gen_random_uuid(),   -- "sid" claim of the access token
  principal_type       identity.principal_type NOT NULL,
  user_id              uuid REFERENCES identity.users(id),
  staff_id             uuid REFERENCES identity.staff_accounts(id),
  device_id            uuid REFERENCES identity.known_devices(id),
  user_agent           varchar(300),
  first_ip             inet,
  approx_location      varchar(60),
  created_at           timestamptz NOT NULL DEFAULT now(),
  last_seen_at         timestamptz NOT NULL DEFAULT now(),
  idle_expires_at      timestamptz NOT NULL,      -- CP-02 / CP-03
  absolute_expires_at  timestamptz NOT NULL,
  ended_at             timestamptz,
  end_reason           identity.session_end_reason,
  CHECK ((principal_type = 'user')  = (user_id  IS NOT NULL)),
  CHECK ((principal_type = 'staff') = (staff_id IS NOT NULL)),
  CHECK (idle_expires_at <= absolute_expires_at),
  CHECK ((ended_at IS NULL) = (end_reason IS NULL))
);
CREATE INDEX sessions_user_active  ON identity.sessions (user_id)  WHERE ended_at IS NULL;
CREATE INDEX sessions_staff_active ON identity.sessions (staff_id) WHERE ended_at IS NULL;

CREATE TABLE identity.refresh_tokens (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id    uuid NOT NULL REFERENCES identity.sessions(id),
  token_hash    bytea NOT NULL UNIQUE CHECK (octet_length(token_hash) = 32),
  issued_at     timestamptz NOT NULL DEFAULT now(),
  used_at       timestamptz,
  replaced_by   uuid REFERENCES identity.refresh_tokens(id) DEFERRABLE INITIALLY DEFERRED,  -- rotation
     inserts the successor after marking this row used
  CHECK ((used_at IS NULL) = (replaced_by IS NULL))
);
-- at most one unused refresh token per session: rotation is strictly linear (SEC-SES-02)
CREATE UNIQUE INDEX refresh_one_live ON identity.refresh_tokens (session_id) WHERE used_at IS NULL;

-- ====================================================================
-- KYC (FR-KYC-01..12, SEC-DAT-02/03, SEC-ACC-06)
-- ====================================================================
CREATE TABLE identity.kyc_submissions (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           uuid NOT NULL REFERENCES identity.users(id),
  doc_type          identity.kyc_doc_type NOT NULL,
  doc_number_enc    bytea,
  doc_number_bidx   bytea CHECK (octet_length(doc_number_bidx) = 32),
  full_name_enc     bytea,
  dob_enc           bytea,
  -- second, non-blocking duplicate signal: HMAC of normalised name + date of birth. One person can
  -- hold NIDs in different number formats, so the document number alone can miss a duplicate.
  person_bidx       bytea CHECK (octet_length(person_bidx) = 32),
  key_version       smallint NOT NULL,
```

```
  is_adult           boolean NOT NULL,                    -- computed from DOB at submission (FR-KYC-03)
  liveness_action    varchar(24) NOT NULL,                -- the random action asked for (FR-KYC-02)
  routed_flags       text[] NOT NULL DEFAULT '{}'         -- reasons the submission needs closer review
                     CHECK (routed_flags <@ ARRAY['duplicate','possible_duplicate','blacklisted','underage','
    precheck']::text[]),
  precheck           jsonb,                               -- advisory only (FR-KYC-11)
  status             identity.kyc_submission_status NOT NULL DEFAULT 'pending_review',
  claimed_by         uuid REFERENCES identity.staff_accounts(id),   -- review assignment (SEC-ACC-06)
  claimed_until      timestamptz,
  decided_by         uuid REFERENCES identity.staff_accounts(id),
  decided_at         timestamptz,
  rejection_reason   identity.kyc_reason,
  rejection_note     varchar(500),
  submitted_at       timestamptz NOT NULL DEFAULT now(),
  erased_at          timestamptz,                         -- retention job (SRS Table 7.2)
  CHECK (erased_at IS NOT NULL OR (doc_number_enc IS NOT NULL AND doc_number_bidx IS NOT NULL
                                   AND full_name_enc IS NOT NULL AND dob_enc IS NOT NULL)),
  CHECK (erased_at IS NULL OR status <> 'pending_review'),
  CHECK ((status = 'pending_review') = (decided_at IS NULL)),
  CHECK ((decided_at IS NULL) = (decided_by IS NULL)),
  CHECK ((status = 'rejected') = (rejection_reason IS NOT NULL)),
  CHECK (status <> 'approved' OR is_adult),
  CHECK ((claimed_by IS NULL) = (claimed_until IS NULL))
);
-- one open submission per user
CREATE UNIQUE INDEX kyc_one_pending ON identity.kyc_submissions (user_id) WHERE status = 'pending_review';
CREATE INDEX kyc_bidx        ON identity.kyc_submissions (doc_number_bidx);
CREATE INDEX kyc_person_bidx ON identity.kyc_submissions (person_bidx);
CREATE INDEX kyc_queue       ON identity.kyc_submissions (submitted_at) WHERE status = 'pending_review';
CREATE INDEX kyc_user_recent ON identity.kyc_submissions (user_id, submitted_at DESC);

CREATE TABLE identity.kyc_files (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  submission_id  uuid NOT NULL REFERENCES identity.kyc_submissions(id),
  kind           identity.kyc_file_kind NOT NULL,
  storage_key    varchar(80) NOT NULL UNIQUE,               -- random name on the encrypted volume
  wrapped_dek    bytea,                                     -- per-file key, wrapped by the KEK; NULL once
    erased
  key_version    smallint NOT NULL,
  sha256         bytea NOT NULL CHECK (octet_length(sha256) = 32),   -- of the re-encoded plaintext
  mime           varchar(20) NOT NULL CHECK (mime IN ('image/jpeg', 'image/png', 'image/webp')),
  byte_size      integer NOT NULL CHECK (byte_size BETWEEN 1 AND 10485760),   -- CP-33
  created_at     timestamptz NOT NULL DEFAULT now(),
  erased_at      timestamptz,                               -- blob deleted and key destroyed
  CHECK ((wrapped_dek IS NULL) = (erased_at IS NOT NULL)),
  UNIQUE (submission_id, kind)
);

-- The guarantee "one identity, one account" (FR-KYC-04) is enforced here:
-- a row is inserted when a submission is approved, inside the same
-- transaction; the primary key makes a second account impossible.
CREATE TABLE identity.identity_claims (
  doc_number_bidx  bytea PRIMARY KEY CHECK (octet_length(doc_number_bidx) = 32),
  user_id          uuid NOT NULL UNIQUE REFERENCES identity.users(id),
  submission_id    uuid NOT NULL REFERENCES identity.kyc_submissions(id),
  claimed_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE identity.kyc_status_history (                                    -- FR-KYC-05, append-only
  id             bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id        uuid NOT NULL REFERENCES identity.users(id),
  from_status    identity.kyc_status NOT NULL,
  to_status      identity.kyc_status NOT NULL,
  actor_type     audit.actor_type NOT NULL,
  actor_id       uuid,
  submission_id  uuid REFERENCES identity.kyc_submissions(id),
  approval_id    uuid,                                      -- FK added below
  reason         text,
  at             timestamptz NOT NULL DEFAULT now(),
  CHECK (from_status <> to_status)
);
CREATE INDEX kyc_hist_user ON identity.kyc_status_history (user_id, at);
CREATE TRIGGER kyc_history_append_only BEFORE UPDATE OR DELETE ON identity.kyc_status_history
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();
```

```
-- ====================================================================
-- Four-eyes approvals (FR-OPS-05, CP-30). Used by every unit; owned here
-- because it is an access-control mechanism over staff identities.
-- ====================================================================
CREATE TABLE identity.approvals (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  action        varchar(60) NOT NULL,          -- e.g. 'kyc.revoke', 'blacklist.add', 'staff.mfa_reset'
  target_type   varchar(40) NOT NULL,
  target_id     uuid NOT NULL,
  payload       jsonb NOT NULL DEFAULT '{}',
  reason        text NOT NULL CHECK (length(btrim(reason)) >= 10),
  required_role identity.staff_role NOT NULL,
  proposed_by   uuid NOT NULL REFERENCES identity.staff_accounts(id),
  proposed_at   timestamptz NOT NULL DEFAULT now(),
  expires_at    timestamptz NOT NULL,
  decided_by    uuid REFERENCES identity.staff_accounts(id),
  decided_at    timestamptz,
  decision      identity.approval_decision,
  decision_note text,
  executed_at   timestamptz,
  CHECK (decided_by IS NULL OR decided_by <> proposed_by),            -- never the same person
  CHECK ((decision IS NULL) = (decided_at IS NULL)),
  -- IS NOT DISTINCT FROM: a plain '=' is NULL when decision is NULL, and a NULL check passes
  CHECK (decision IS NOT DISTINCT FROM 'expired' OR (decision IS NULL) = (decided_by IS NULL)),
  CHECK (executed_at IS NULL OR decision IS NOT DISTINCT FROM 'approved')
);
CREATE INDEX approvals_open ON identity.approvals (proposed_at) WHERE decision IS NULL;
-- only one open proposal per action and target
CREATE UNIQUE INDEX approvals_one_open ON identity.approvals (action, target_id) WHERE decision IS NULL;

ALTER TABLE identity.kyc_status_history
  ADD FOREIGN KEY (approval_id) REFERENCES identity.approvals(id);

-- ====================================================================
-- NID blacklist (FR-TNS-06, FR-GAM-18). Entries are added or removed only
-- through an executed approval.
-- ====================================================================
CREATE TABLE identity.nid_blacklist (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  doc_number_bidx   bytea NOT NULL CHECK (octet_length(doc_number_bidx) = 32),
  source            varchar(30) NOT NULL CHECK (source IN ('game_account_recovery', 'fraud_decision')),
  reason            text NOT NULL,
  related_order_id  uuid,                  -- order unit; no cross-unit FK (DC-5)
  added_by          uuid NOT NULL REFERENCES identity.staff_accounts(id),
  add_approval_id   uuid NOT NULL REFERENCES identity.approvals(id),
  added_at          timestamptz NOT NULL DEFAULT now(),
  removed_at        timestamptz,
  removed_by        uuid REFERENCES identity.staff_accounts(id),
  remove_approval_id uuid REFERENCES identity.approvals(id),
  CHECK ((removed_at IS NULL) = (remove_approval_id IS NULL)),
  CHECK ((removed_at IS NULL) = (removed_by IS NULL))
);
CREATE UNIQUE INDEX blacklist_active ON identity.nid_blacklist (doc_number_bidx) WHERE removed_at IS NULL;

-- ====================================================================
-- Audit log (SEC-LOG-01..04). Shared by all units; tamper-evident chain.
-- ====================================================================
CREATE TABLE audit.events (
  id            bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  -- position in the hash chain, assigned under the chain lock. Ids are allocated before the lock,
  -- so under concurrent inserts id order and chain order can differ; verification uses chain_pos.
  chain_pos     bigint NOT NULL UNIQUE,
  at            timestamptz NOT NULL DEFAULT clock_timestamp(),
  actor_type    audit.actor_type NOT NULL,
  actor_id      uuid,
  action        varchar(80) NOT NULL CHECK (action ~ '^[a-z]+(\.[a-z_]+)+$'),   -- 'kyc.file.viewed'
  object_type   varchar(40),
  object_id     uuid,
  before_state  jsonb,
  after_state   jsonb,
  reason        text,
  approval_id   uuid,
  source_ip     inet,
  correlation_id uuid,
  prev_hash     bytea NOT NULL,
```

```
  hash            bytea NOT NULL UNIQUE
);
CREATE INDEX audit_object ON audit.events (object_type, object_id, at);
CREATE INDEX audit_actor  ON audit.events (actor_type, actor_id, at);

-- The chain: hash = SHA-256(prev_hash || canonical text of the row).
-- An advisory lock serialises inserts so that the chain has no forks.
CREATE FUNCTION audit.chain_hash() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE
  last_hash bytea;
  last_pos  bigint;
BEGIN
  PERFORM pg_advisory_xact_lock(7461001);           -- constant key for the audit chain
  SELECT e.hash, e.chain_pos INTO last_hash, last_pos FROM audit.events e ORDER BY e.chain_pos DESC LIMIT 1;
  NEW.chain_pos := coalesce(last_pos, 0) + 1;
  NEW.at        := clock_timestamp();               -- time taken under the lock, so it never goes backwards
  NEW.prev_hash := coalesce(last_hash, '\x00'::bytea);
  NEW.hash := digest(
      NEW.prev_hash ||
      convert_to(concat_ws('|', NEW.chain_pos, to_char(NEW.at AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.US
    '),
                 NEW.actor_type, NEW.actor_id, NEW.action, NEW.object_type, NEW.object_id,
                 NEW.before_state::text, NEW.after_state::text, NEW.reason, NEW.approval_id,
                 host(NEW.source_ip), NEW.correlation_id), 'UTF8'),
      'sha256');
  RETURN NEW;
END $$;
CREATE TRIGGER audit_chain BEFORE INSERT ON audit.events
  FOR EACH ROW EXECUTE FUNCTION audit.chain_hash();
CREATE TRIGGER audit_append_only BEFORE UPDATE OR DELETE ON audit.events
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

-- Verification used by the scheduled chain check (SEC-LOG-02): returns the id of
-- the first entry (in chain order) whose hash does not match, or NULL if intact.
-- A deleted entry also breaks the chain, because its successor's prev_hash no longer matches.
CREATE FUNCTION audit.verify_chain() RETURNS bigint LANGUAGE plpgsql STABLE AS $$
DECLARE
  r audit.events;
  expected_prev bytea := '\x00'::bytea;
  calc bytea;
BEGIN
  FOR r IN SELECT * FROM audit.events ORDER BY chain_pos LOOP
    IF r.prev_hash <> expected_prev THEN RETURN r.id; END IF;
    calc := digest(
      r.prev_hash ||
      convert_to(concat_ws('|', r.chain_pos, to_char(r.at AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.US'),
                 r.actor_type, r.actor_id, r.action, r.object_type, r.object_id,
                 r.before_state::text, r.after_state::text, r.reason, r.approval_id,
                 host(r.source_ip), r.correlation_id), 'UTF8'),
      'sha256');
    IF calc <> r.hash THEN RETURN r.id; END IF;
    expected_prev := r.hash;
  END LOOP;
  RETURN NULL;
END $$;

-- ================================================================
-- Transactional outbox (design convention DC-6). Events that must not be
-- lost (security notices, KYC decisions) are written in the same
-- transaction as the change; a worker publishes them and marks them sent.
-- ================================================================
CREATE TABLE identity.outbox (
  id             bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_type     varchar(80) NOT NULL CHECK (event_type ~ '^identity\.[a-z_]+\.[a-z_]+$'),
  aggregate_id   uuid NOT NULL,
  payload        jsonb NOT NULL,
  created_at     timestamptz NOT NULL DEFAULT now(),
  attempts       smallint NOT NULL DEFAULT 0,
  next_attempt_at timestamptz NOT NULL DEFAULT now(),
  dispatched_at  timestamptz
);
CREATE INDEX outbox_pending ON identity.outbox (next_attempt_at) WHERE dispatched_at IS NULL;

-- ================================================================
-- Rule: kyc_status may change only along permitted edges (FR-KYC-05).
```

```
-- The application writes kyc_status_history in the same transaction;
-- this trigger is the last line of defence.
-- ====================================================================
CREATE FUNCTION identity.check_kyc_transition() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.kyc_status IS DISTINCT FROM OLD.kyc_status THEN
    IF NOT ((OLD.kyc_status, NEW.kyc_status) IN (
        ('unsubmitted'::identity.kyc_status, 'pending_review'::identity.kyc_status),
        ('rejected',       'pending_review'),
        ('pending_review', 'verified'),
        ('pending_review', 'rejected'),
        ('verified',       'revoked'))) THEN
      RAISE EXCEPTION 'kyc_status␣%␣->␣%␣is␣not␣permitted', OLD.kyc_status, NEW.kyc_status
        USING ERRCODE = 'check_violation';
    END IF;
    NEW.kyc_status_changed_at := now();
  END IF;
  NEW.updated_at := now();
  RETURN NEW;
END $$;
CREATE TRIGGER users_kyc_transition BEFORE UPDATE ON identity.users
  FOR EACH ROW EXECUTE FUNCTION identity.check_kyc_transition();

-- ====================================================================
-- Roles and grants (SEC-INF-03, SEC-LOG-04)
-- ====================================================================
DO $$ BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'tb_app') THEN
    CREATE ROLE tb_app LOGIN;
  END IF;
END $$;
GRANT USAGE ON SCHEMA identity, audit TO tb_app;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA identity TO tb_app;
GRANT DELETE ON identity.pending_registrations, identity.otp_challenges,
               identity.one_time_tokens, identity.staff_conflicts TO tb_app;  -- housekeeping only
REVOKE UPDATE ON identity.kyc_status_history FROM tb_app;
GRANT SELECT, INSERT ON audit.events TO tb_app;                               -- no UPDATE / DELETE
GRANT EXECUTE ON FUNCTION audit.verify_chain() TO tb_app;
```